



计算机图形学基础

胡事民

清华大学计算机科学与技术系



本节课程内容

- 纹理简述
- 纹理采集、纹理合成
- 纹理映射
- 纹理滤波
- 纹理相关的高级主题



纹理 (Texture) 简述

- 纹理

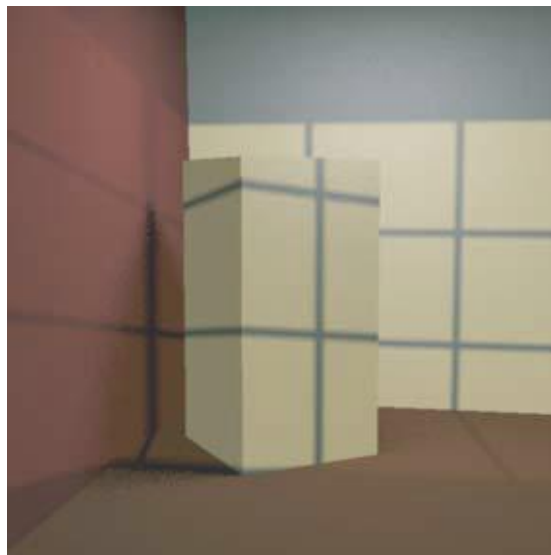
- 计算机图形学的一个基本概念。
- 它使得人们不必对模型的几何细节进行建模，就可以表示丰富的表面细节。
- 这次课我们将讲述纹理的基本思想和应用。



纹理 (Texture)

- 纹理

- 请注意左图(无纹理)和右图(有纹理)的区别。
- 右图更加逼真 (**realistic**)。





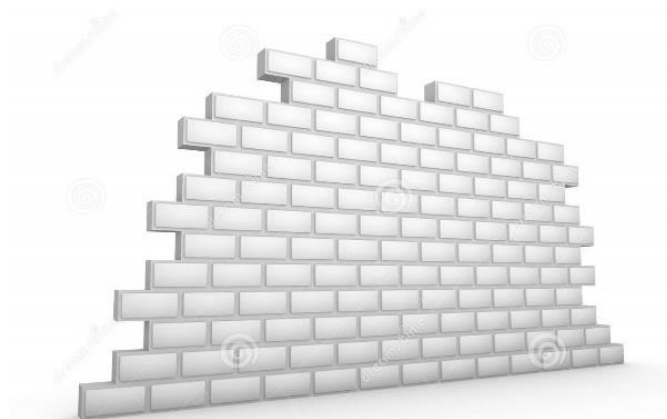
纹理 (Texture)

- 为什么我们需要纹理?

- 绘制真实感图像的最重要的任务之一，就是对表面细节进行建模。

- 例如，如果我们想对一块砖墙进行建模。

- 选择之一：用大量的具有恰当表面颜色和反射属性的多边形对表面细节进行建模。





纹理 (Texture)

- 为什么我们需要纹理?

- 然而，在大多数应用中，很少有人会去注意太多砖墙的细节。大多数人是在一定距离以外观看这面砖墙。
- 在这种情况下，我们可能并不需要知道砖墙的所有细节。相反，我们可以用大的多边形平面对墙进行建模，并把一幅墙的图片贴在多边形上以使得这个多边形看起来像一面砖墙。





纹理 (Texture)

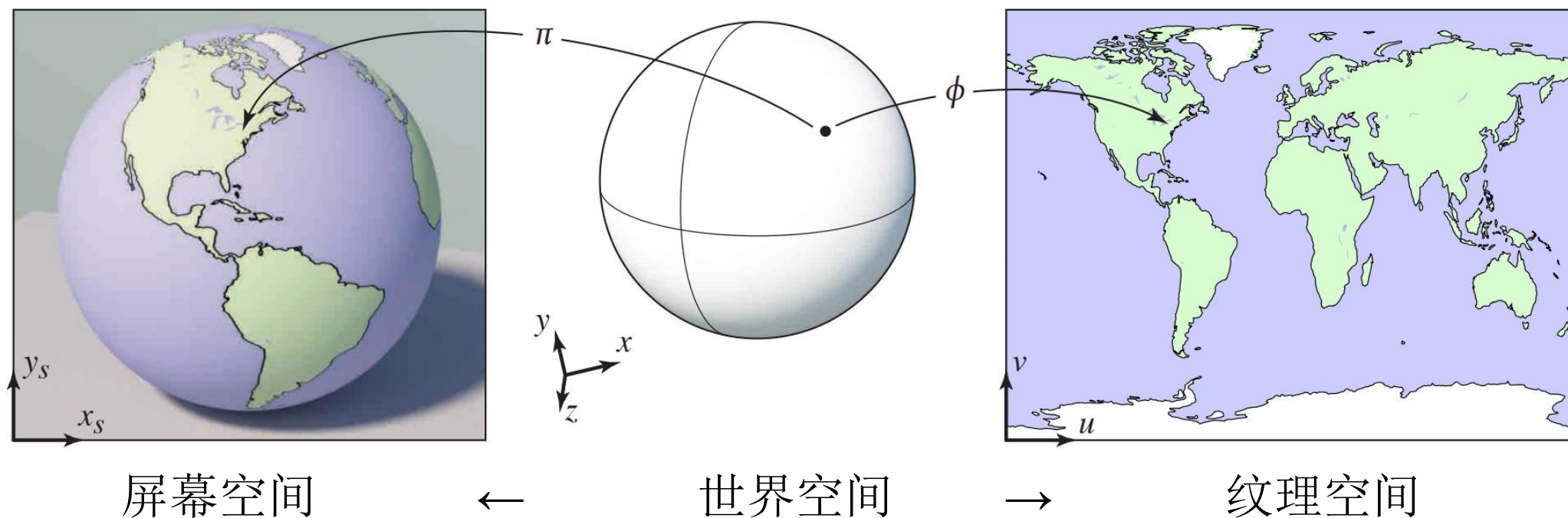
- 什么是纹理?
 - 这个例子中，用于近似真实砖墙的图像，在图形学中即被称为纹理。
 - 将纹理应用于物体表面的过程被称为纹理映射(texture mapping)或贴纹理(texturing)。



纹理 (Texture)

- 什么是纹理?

- 纹理本质上是建立二维**纹理贴图**与三维**物体表面**的坐标映射





纹理 (Texture)

- 为什么纹理是有用的?

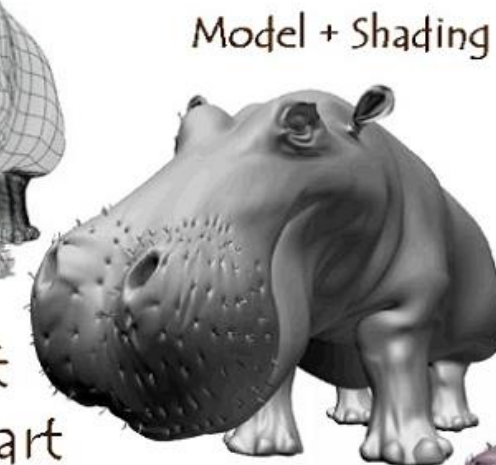
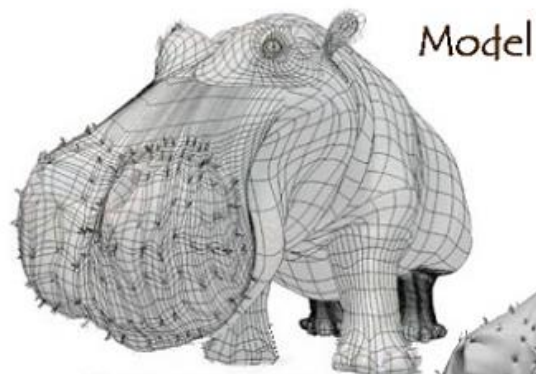
- 纹理解决了两个问题.

- 首先, 提高建模效率; 通过纹理图像, 用户不必建模所有的几何及材质细节, 使得用户可以将精力集中于其它更重要的事情上.
 - 其次, 提高渲染效率; 通过渲染简单的多边形模型 (例如: 用一个单一的多边形表示砖墙) 和纹理, 而非具有不同BRDF的模型几何细节, 这种改进, 可节约计算时间和计算资源.



纹理 (Texture)

- 为什么我们需要纹理?



At what point
do things start
looking real?

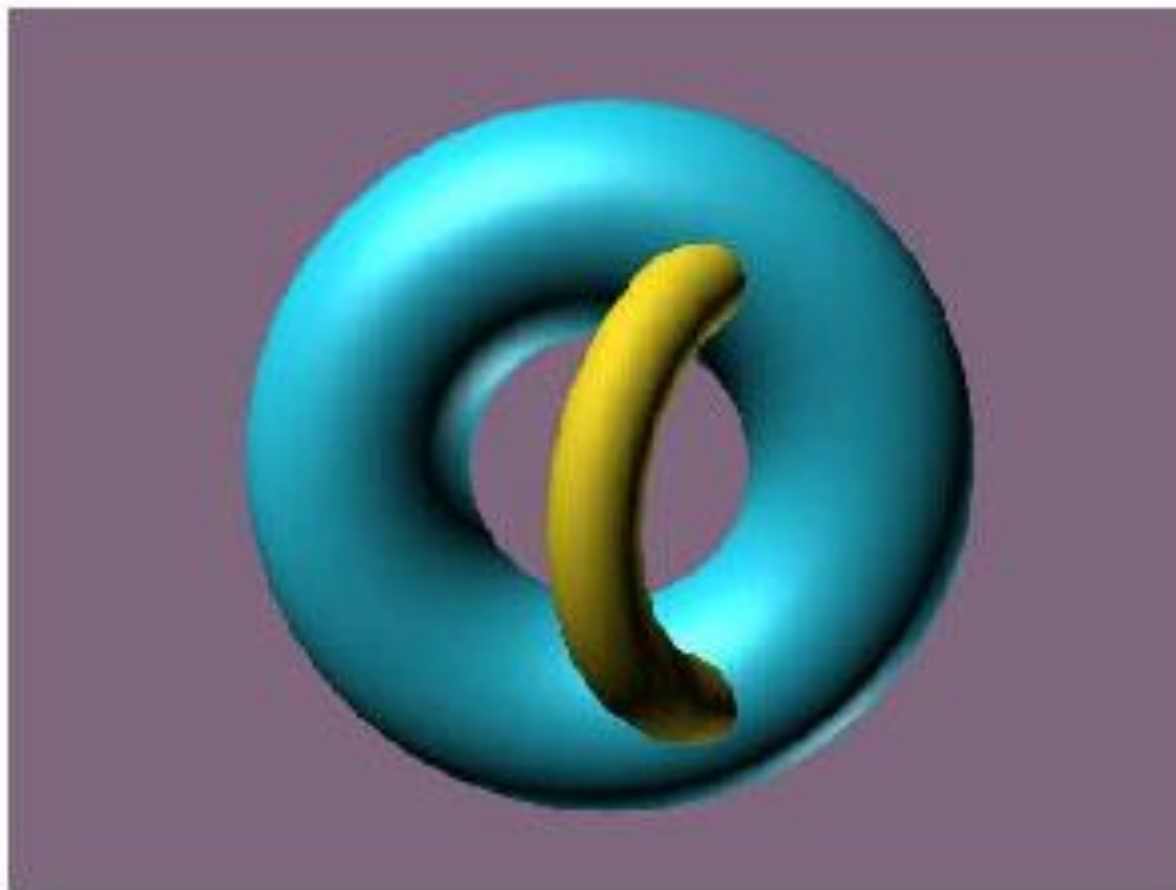


For more info on the computer artwork of Jeremy Birn
see <http://www.3drender.com/jbirn/productions.html>

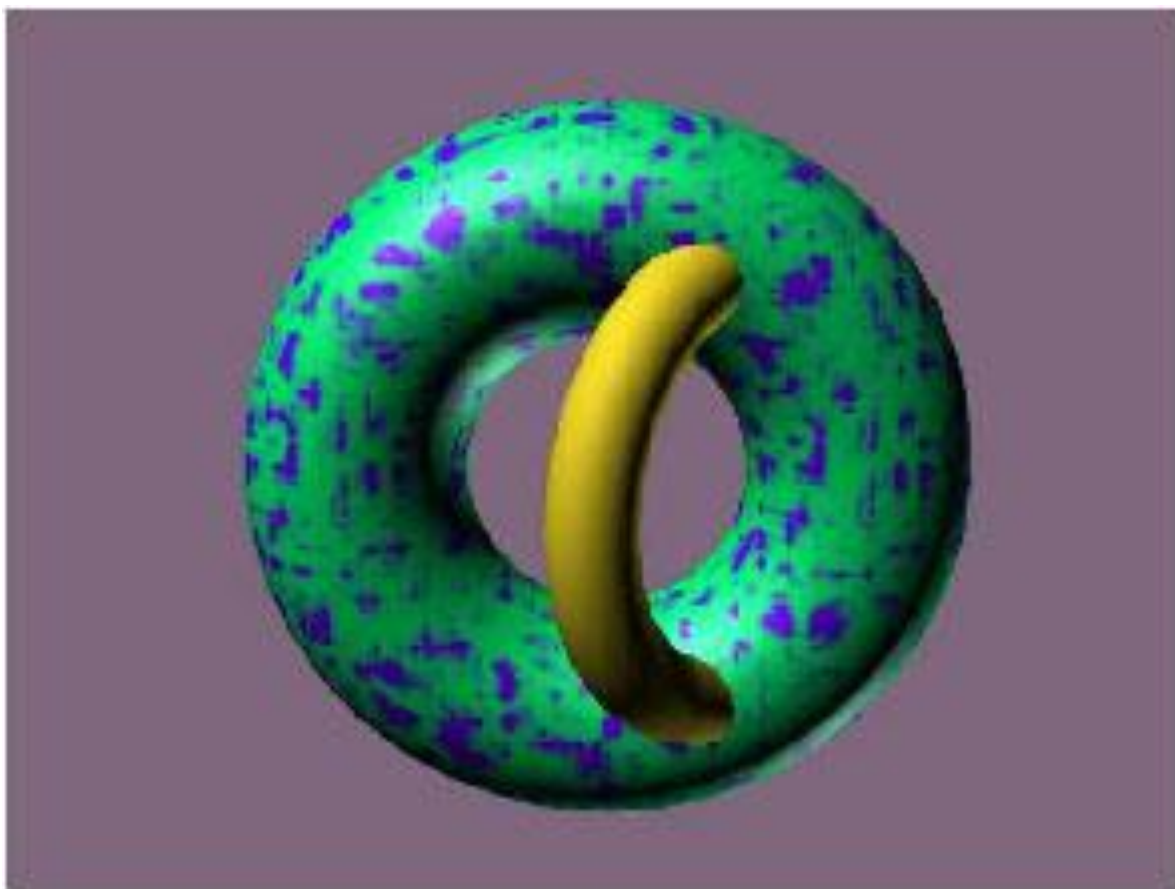


更多的纹理示例

原始模型

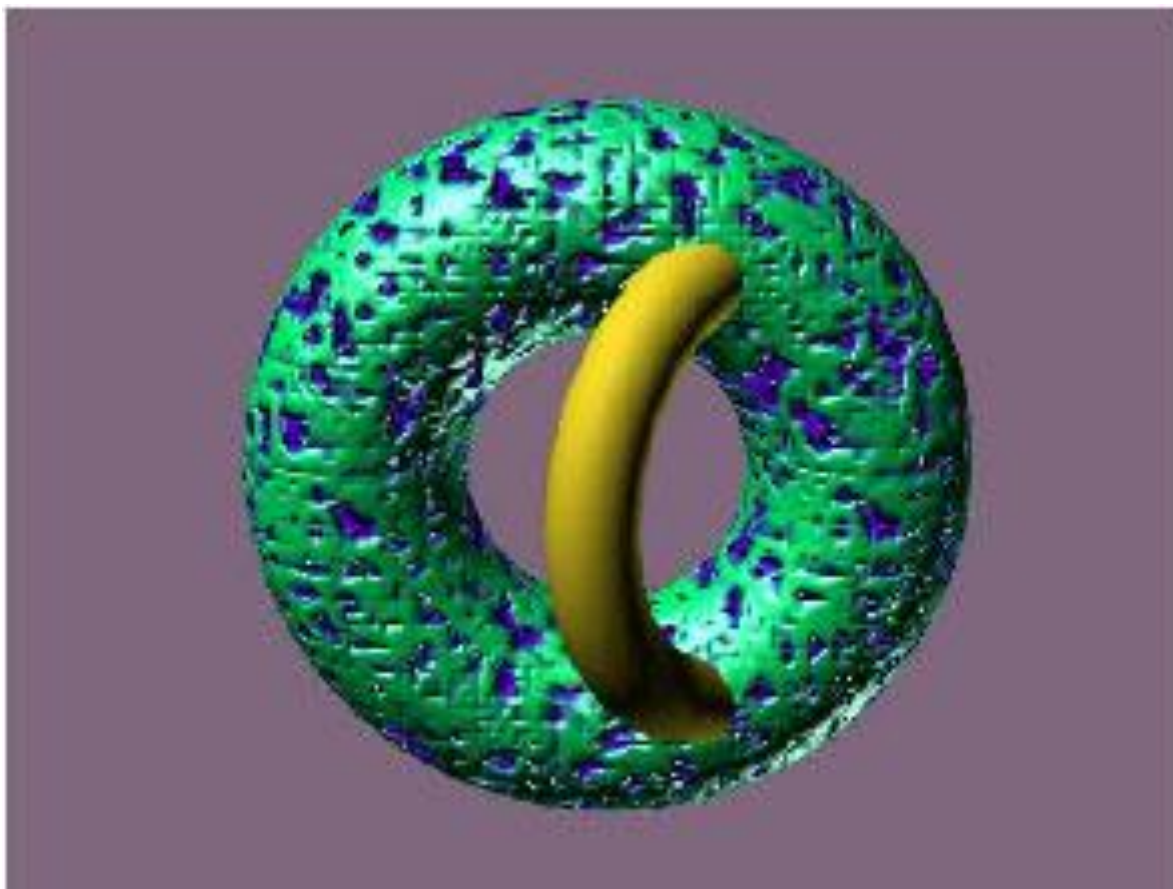


二维纹理

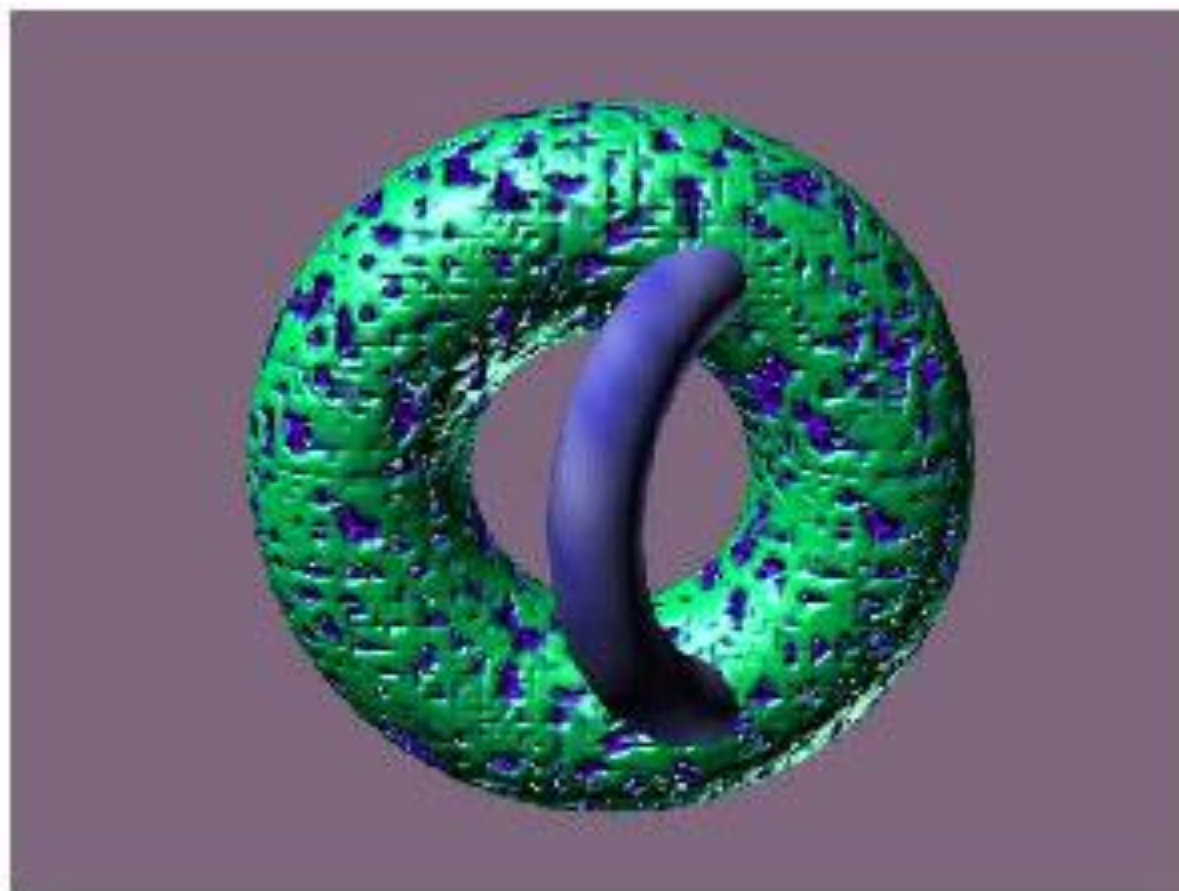




几何纹理 (Bump texture)



组合



模型及纹理图





贴过纹理的模型



另一个图像





贴了新的纹理





贴了另外一个纹理





使用几何纹理





纹理(Texture)

- 纹理的使用

- 为了使用纹理，我们通常需要三个步骤

- 1) 纹理采集(acquisition)。

- 这个步骤可以通过以下方式完成：

- 手工绘制(manually drawing)

- 拍照(taking photographs)

- 过程纹理(procedure texture)

- 纹理合成 (texture synthesis)



纹理(Texture)

- 纹理的使用

- 为了使用纹理，我们通常需要三个步骤

- 1) 纹理采集(acquisition)。

- 2) 纹理贴图(Texture mapping)

- 给定纹理图像和一个三维模型，如何将纹理映射到模型上？

- 可以通过给每个三维模型的顶点指定一个纹理坐标，并利用这个坐标把纹理图像上的色彩赋值给模型来实现。



纹理(Texture)

- 纹理的使用

- 为了使用纹理，我们通常需要三个步骤

- 1) 纹理采集(acquisition)
- 2) 纹理贴图(Texture mapping)
- 3) 纹理滤波(texture filtering)

在决定如何将纹理映射到物体上去之后，绘制时还需仔细地对纹理进行采样。否则，容易产生一些不好的人为痕迹(artifacts)，这种问题的产生源于信号处理理论中的一些基本问题。



本节课程内容

- 纹理简述
- 纹理采集、纹理合成
- 纹理映射
- 纹理滤波
- 纹理相关的高级主题
- 卷积神经网络基础



纹理采集 (acquisition)

- 纹理映射的第一个任务就是纹理采集（如何得到纹理贴图）。采集纹理图像的方式多种多样。
 - 可以用手工绘制纹理。
 - 可以简单地采集待建模物体的照片。
 - 我们不把重点放在上述两种方法，而讨论更为技术性的两种方法：
 - 过程纹理(procedure texture)
 - 纹理合成(texture synthesis)



过程纹理(Procedure Texture)

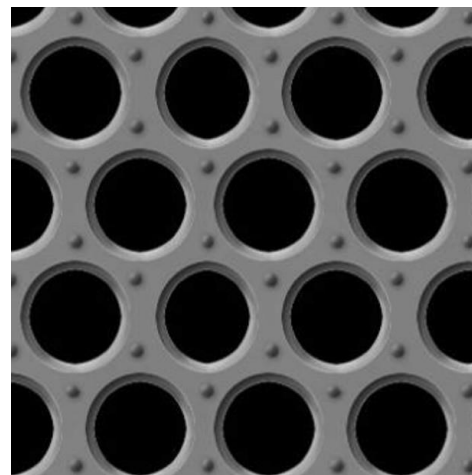
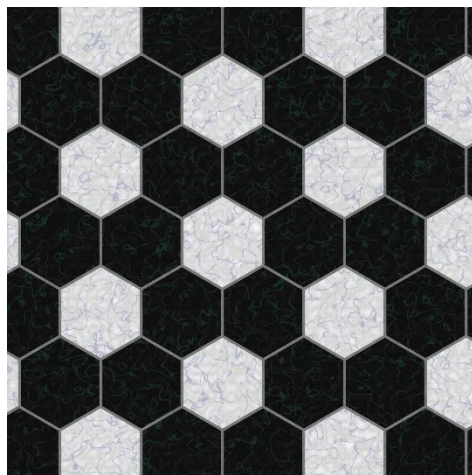
- 通过编写特殊的程序模拟物理形成过程或者材质外观，来合成纹理。
- 例如，像大理石或者木头等一些特定的模式，可以通过非常简单的函数来模拟。



marble



wood

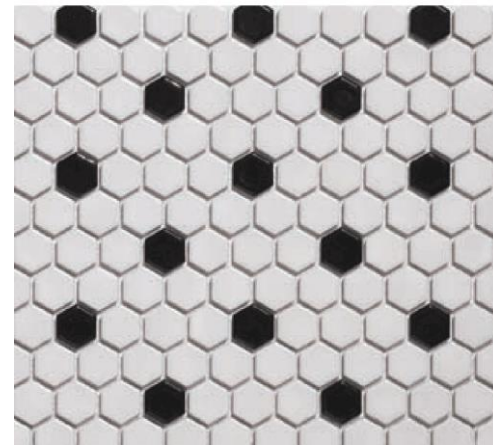
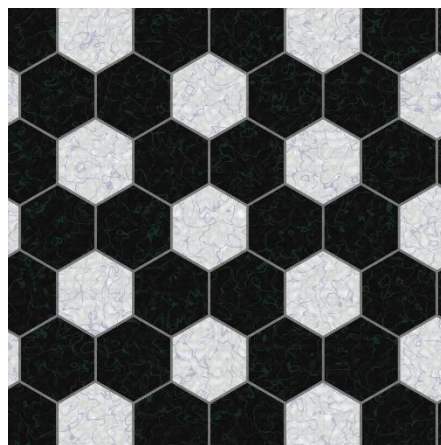




过程纹理(Procedure Texture)

- 优点

- 这种表示可以非常紧致，因为只需存储过程函数和参数。
- 通过改变过程参数可以简单地改变材质的外观，提供优良的可控性。





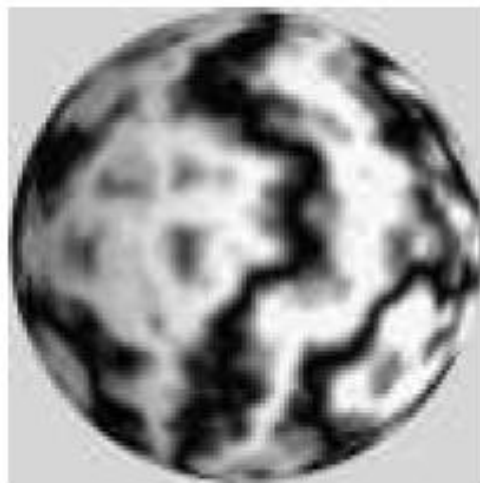
过程纹理(Procedure Texture)

- 缺点
 - 然而，过程式合成(procedural synthesis)方法只能被用于一些特定纹理的生成。
 - 对于一个没有已知程序代码的材质，无法通过该方法合成。



过程纹理(Procedure Texture)

- 最著名的过程式合成方法: 柏林噪声 [Perlin2002]



marble



wood



过程纹理(Procedure Texture)

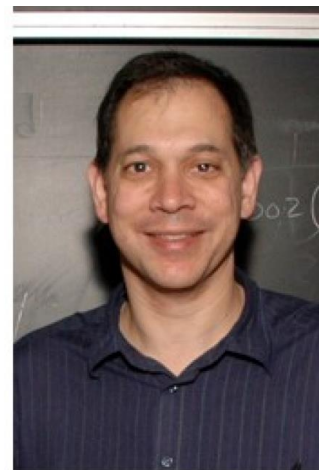
- 柏林噪声(Perlin Noise)

- 从1985年起，柏林噪声就成为一种广泛应用的标准过程纹理产生方法。
- 柏林噪声的基本思路简单且实用。在介绍柏林噪声前，我们将首先描述白噪声。

- Kenneth Perlin

ACM SIGGRAPH Achievement Award

2008





过程纹理(Procedure Texture)

- 白噪声 (White Noise)

- 白噪声是一种在所有频率处具有均匀能量分布的信号。因此，白噪声的傅里叶变换是近似平坦的。
- 可以通过均匀的随机函数产生。
- 例如，要产生一个二维的白噪声图像，可以用 $[0, 1]$ 区间内的均匀分布随机函数产生的值作为每个像素的颜色。



过程纹理(Procedure Texture)

- 柏林噪声(Perlin Noise)

- 和白噪声不同，柏林噪声是一个有限频宽函数(band limited function)。它可以通过对不同频带(frequency bands)的白噪声求和得到。

$$perlin = \sum_{i=0}^{n-1} interpolate(white_i) \times p^i$$

- n 是频带总数目， p 表示持久性 (persistence), i 是频带编号，其中 $i=0$ 代表最低频带。



过程纹理(Procedure Texture)

- 柏林噪声

$$perlin = \sum_{i=0}^{n-1} interpolate(white_i) \times p^i$$

- 上式中，第*i*个频带的白噪声是一个简单的具有特定大小(2^i)的白噪声。由于不同频带对应不同的图像大小，在求和前需要进行插值(interpolation)。
- 持久性是一个用户指定的参数。通过它可以简单的控制不同频带之间的相对权重。这个参数范围通常是 $[0,1]$.



过程纹理(Procedure Texture)

- 柏林噪声

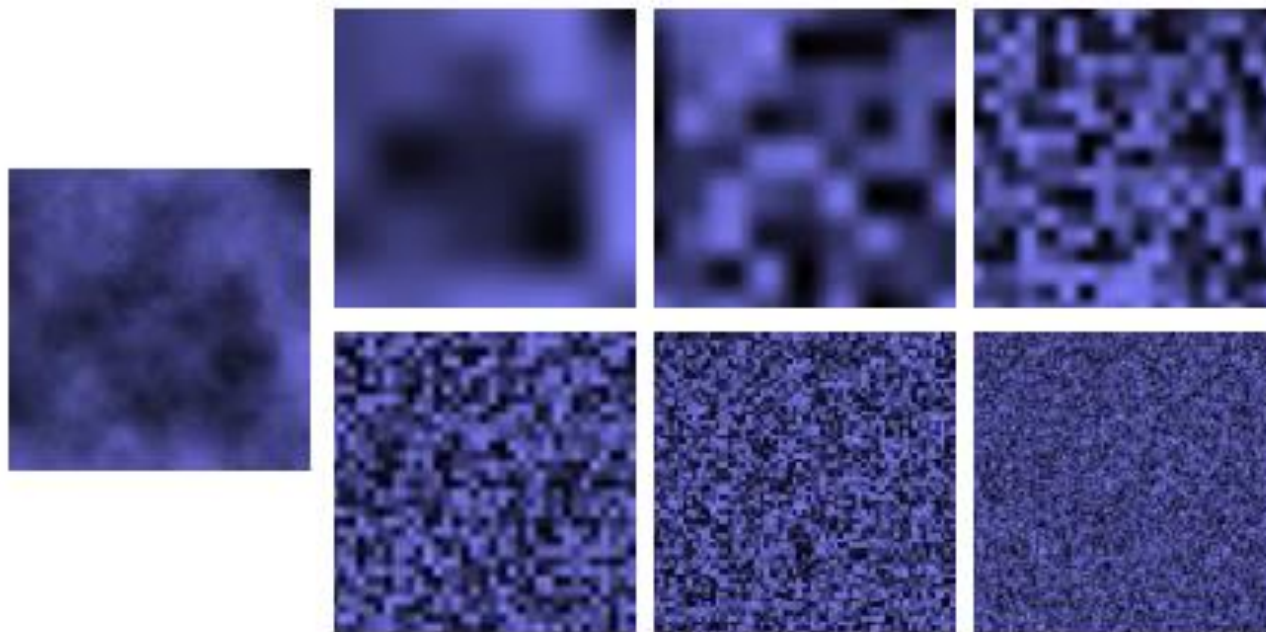


Figure 1: 2D Perlin noise example. Left: the Perlin noise image. Right: the individual noise bands, from low to high frequencies. Image courtesy of [Elias 2003].



过程纹理(Procedure Texture)

- 柏林噪声

- 基于柏林噪声，可以利用适当的过程生成丰富多样的纹理。
例如生成大理石纹理和木材纹理。对应的公式如下：

$$marble = \cos(x + perlin(x, y, z))$$

$$g = perlin(x, y, z) * scale$$

$$wood = g - \text{int}(g)$$



marble



wood



纹理合成

- 什么是纹理合成?
 - 从一个特定的样本合成一块大的纹理。很明显，这种方式很受用户喜欢。因为使用该算法，用户仅需提供一个很小的图像样本。
 - 由于图形学和计算机视觉领域长期以来有很多研究致力于纹理分析，已有许多实用的算法。



纹理合成

- 在计算机图形学中，纹理合成技术可以被分为两类：
 - 基于像素(Pixel-based)的纹理合成
 - 基于块 (Patch-based)的纹理合成

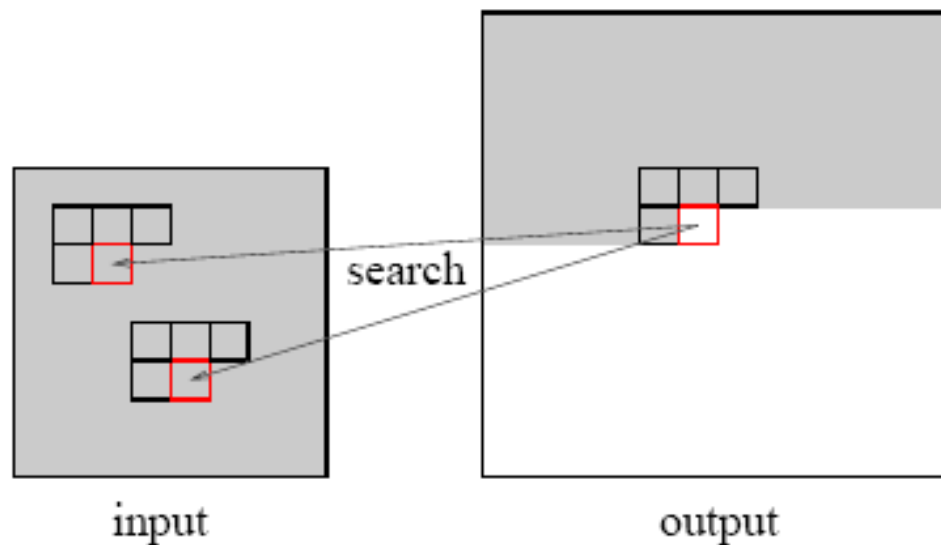


纹理合成

- 基于像素的（**pixel based**）纹理合成

- 主要思想

- 逐像素地合成新的纹理，其中每个像素的值由局部邻域来确定，(例如： 3×3 , 5×5)，选择输入像素中具有最相似邻域的那个。



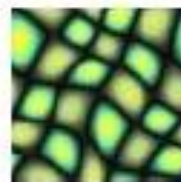


纹理合成

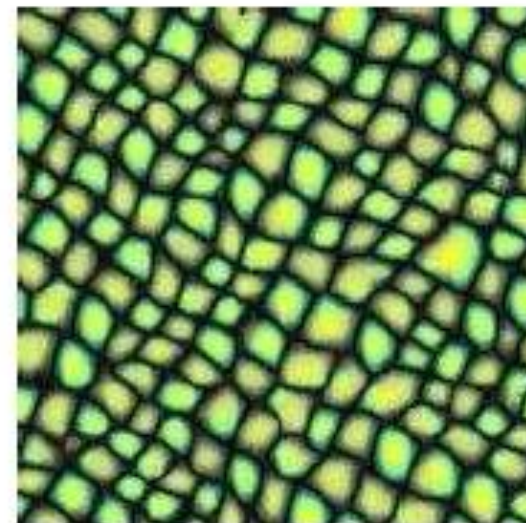
- 基于像素的纹理合成

- 这类工作包括:

- “Texture Synthesis by Non-parametric Sampling” [Efros99] (4860 cites)
 - “Fast texture synthesis using tree-structured vector quantization” [Wei and Levoy00] (2219 cites)



sample



result

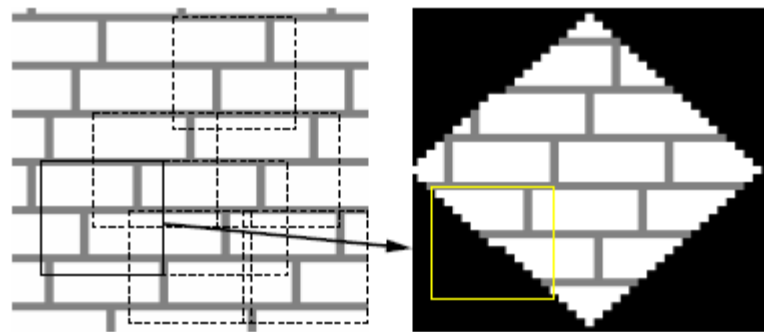


纹理合成

- **Texture Synthesis by Non-parametric Sampling**

- 每次合成一个像素(i, j);
- 将以(i, j)为中心点, 宽度为 w 的块与纹理样本中同样大小的块进行比对; (w 通常取5,11,15等)
- 将其中一个相似块的中心点像素值赋给(i, j)。

- 如何度量块的相似(差异)度?





纹理合成

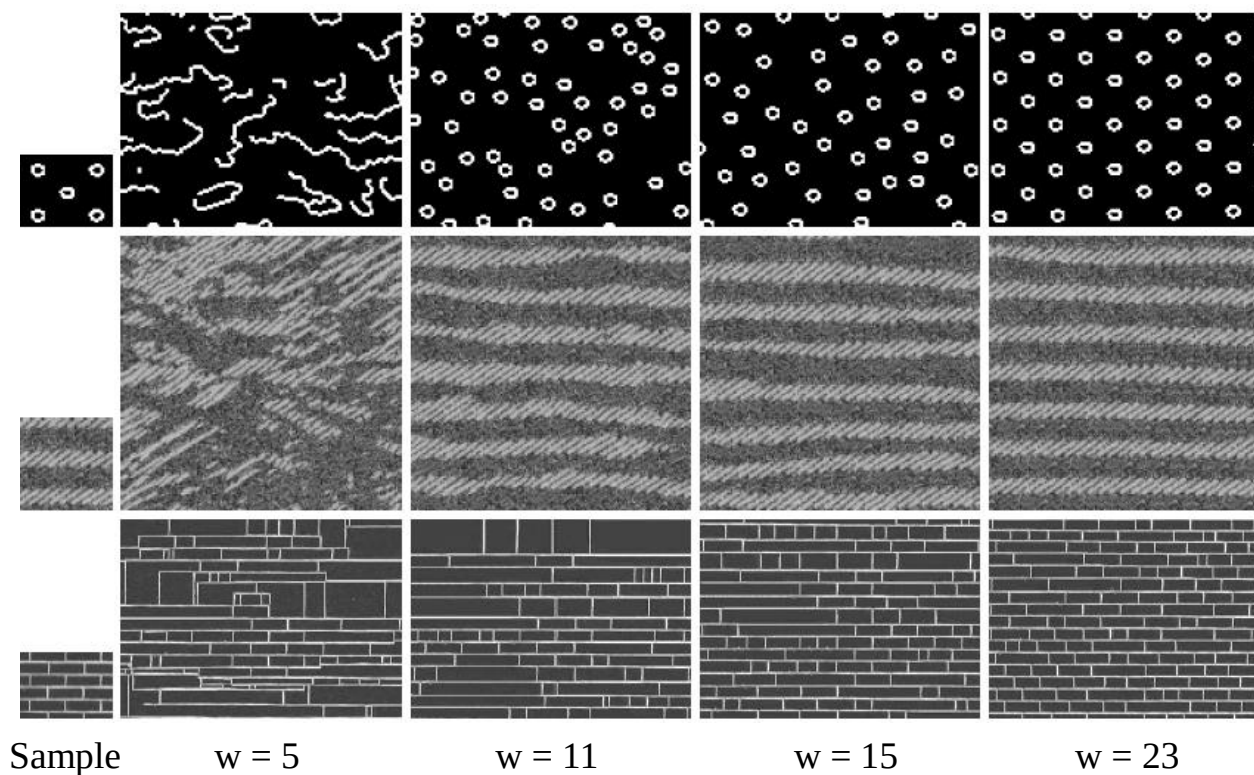
- **Texture Synthesis by Non-parametric Sampling**

- 块的差异度 $d = \sum \|(\text{block} - \text{kernel}) \cdot G\|^2$;
 - G 为同等大小的高斯核（或等值核），只计算有效像素。
- 所有可选取的相似块满足： $d < (1 + \xi) \cdot d_{min}$;
 - 论文中 $\xi = 0.1$ 。
- 随机选取一块相似块。



纹理合成

- **Texture Synthesis by Non-parametric Sampling**
 - 不同 w 的实验结果

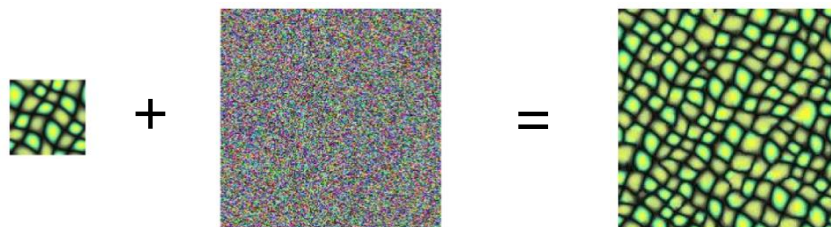




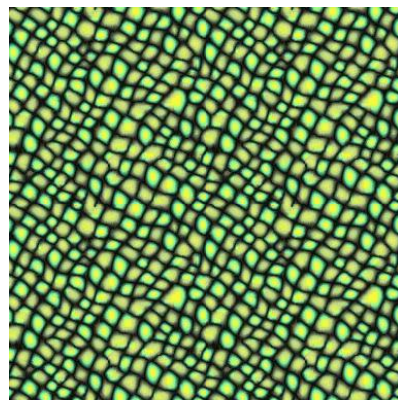
纹理合成

- **Fast Texture Synthesis using Tree-structured Vector Quantization**

- 通过纹理元和一张随机白噪声图像合成纹理。



- 这种合成纹理可平铺(tileable)



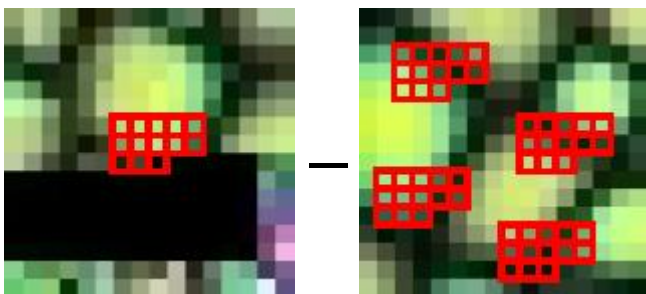
魏立一的故事



纹理合成

- **Fast Texture Synthesis using Tree-structured Vector Quantization**

- 使用光栅扫描顺序进行像素填充；
- 使用L形邻域，计算方法和上篇相同：

$$d = \sum \left\| \left(\begin{array}{c} \text{Image 1} \\ \text{Image 2} \end{array} - \begin{array}{c} \text{Image 3} \\ \text{Image 4} \end{array} \right) \cdot G \right\|^2$$


- 循环处理边界像素，从而保证可平铺性。

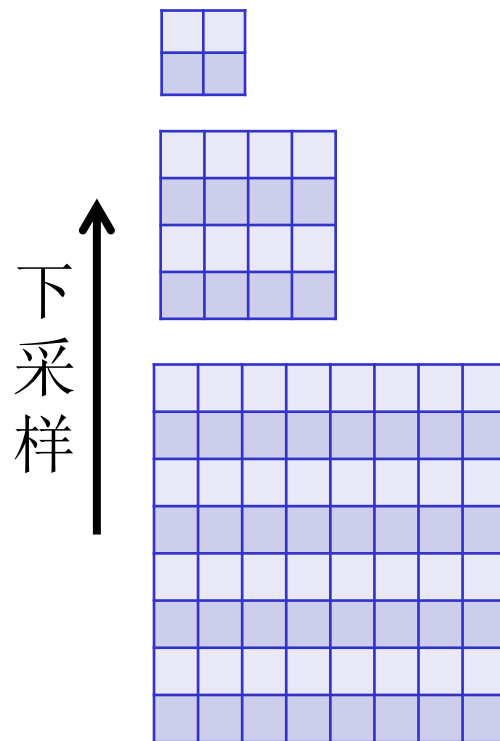




纹理合成

- **Fast Texture Synthesis using Tree-structured Vector Quantization**

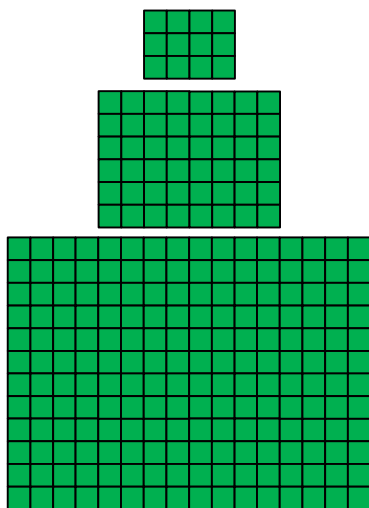
- 算法速度取决于使用的块的大小 w ;
对含大尺度结构的纹理, 需用大的块。
- 为了加速合成过程, 文章给出了一种使用小块合成大尺度纹理的方法: 引入图像金字塔。



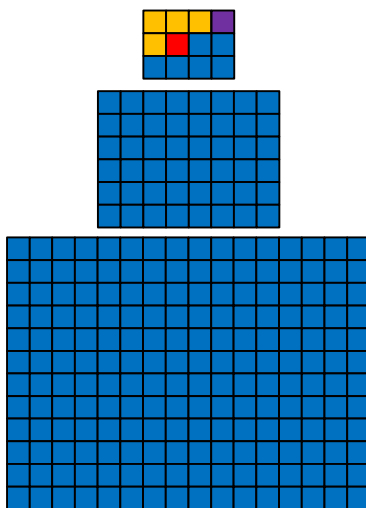


纹理合成

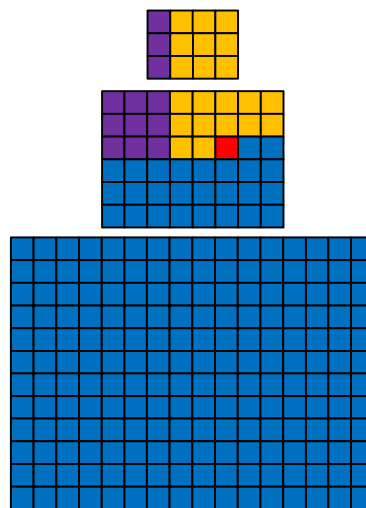
- **Fast Texture Synthesis using Tree-structured Vector Quantization**
 - 在逐级合成的过程中，给后一级的像素赋值时，同时计算两级的误差，并选用误差和最小的像素赋值。



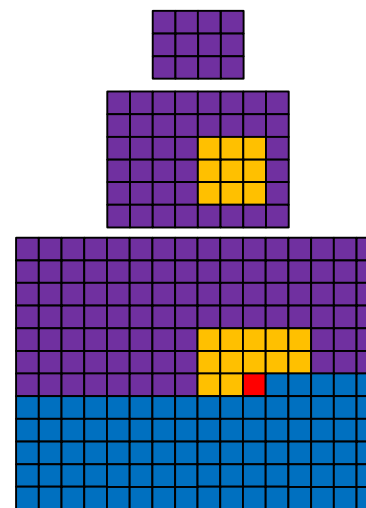
原纹理金字塔



处理第1层



处理第2层

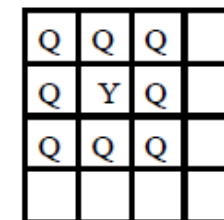
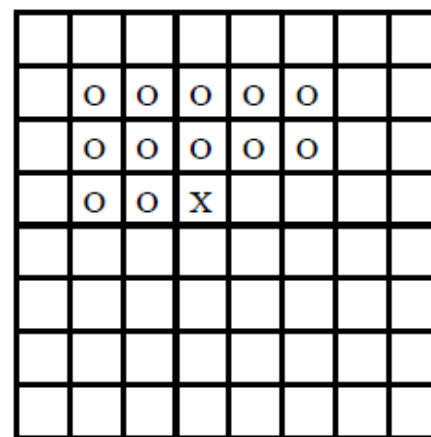
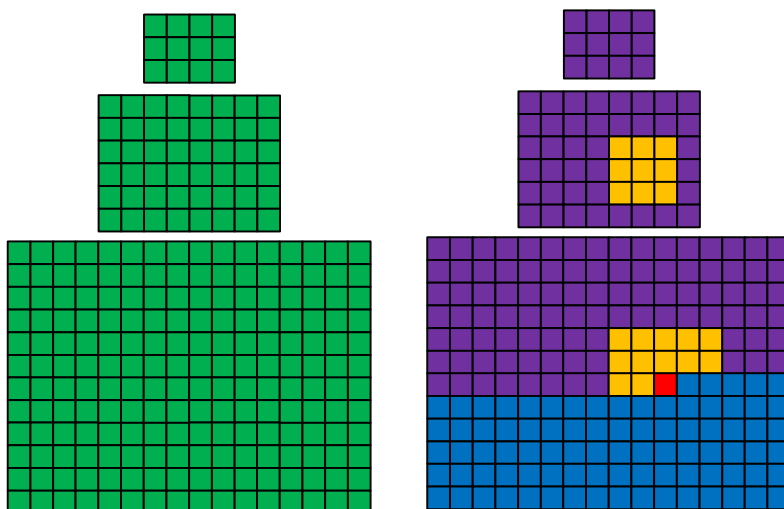


处理第3层



纹理合成

- **Fast Texture Synthesis using Tree-structured Vector Quantization**
 - 给X赋值时，找出它在前一级中的对应像素Y，最终的候选块应该和O、Q两层的误差和最小。





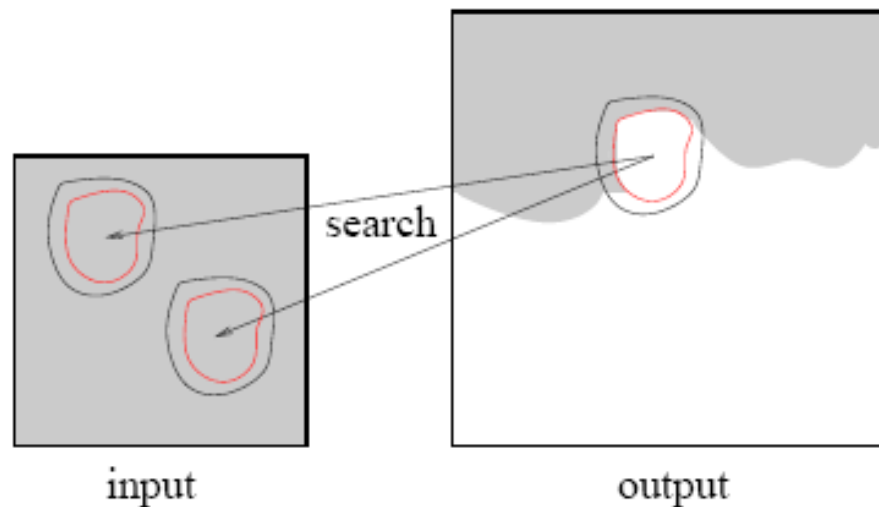
纹理合成

- 基于块的（**Region based**）纹理合成

- 基于像素的方法可以通过合成区块来改进。

- 最重要的方法之一是 **Graph-cut[Kwatra03]**, 该方法是目前为止效果最好的。

- 区块的选择同样是根据邻域匹配来进行的。



Vivek Kwatra, Arno Schödl, Irfan Essa, Greg Turk, Aaron F Bobick, Graphcut textures: image and video synthesis using graph cuts, ACM SIGGRAPH 2003. (2191 cites)

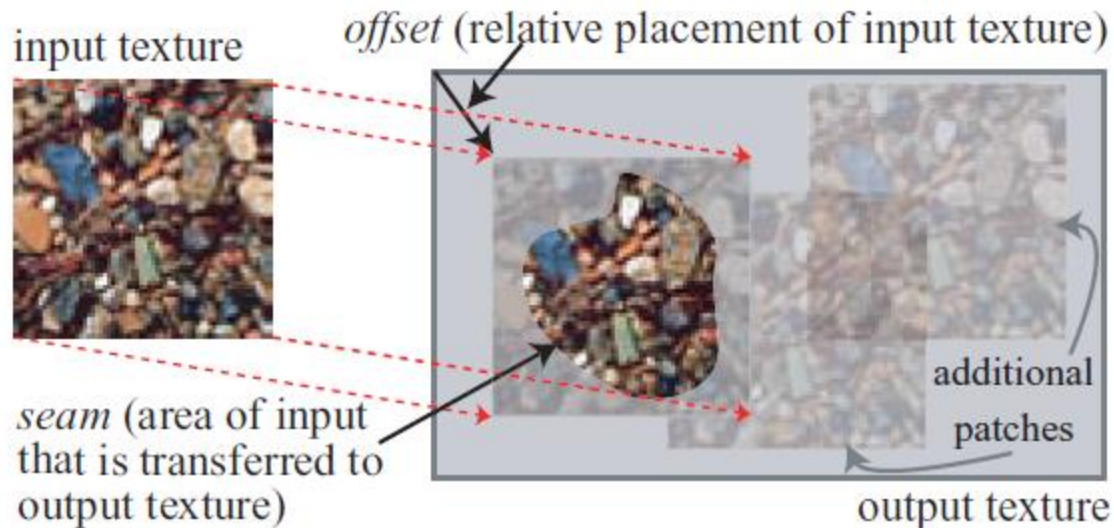


纹理合成

- **Graphcut Textures: Image and Video Synthesis Using Graph Cuts**

- 初始化：将一份input随机置于output中
- 更新分为两个步骤：

- 第一步：在output中选取合适的偏移量放入input;
- 第二步：决定重叠部分使用新的input还是原来的值





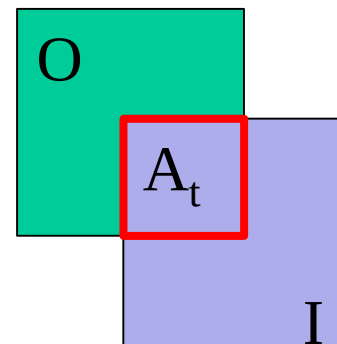
纹理合成

- **Graphcut Textures: Image and Video Synthesis Using Graph Cuts**

- 第一步：计算候选块的偏移量，有三种候选方法：

- 1. 随机放置：随机放在一个地方，适用于随机纹理。
- 2. 全块差异度量：计算使得重叠区域对应像素距离平方和最小的位置，适用于结构化纹理：

$$C(t) = \frac{1}{|A_t|} \sum_{p \in A_t} |\mathbf{I}(p) - \mathbf{O}(p+t)|^2$$





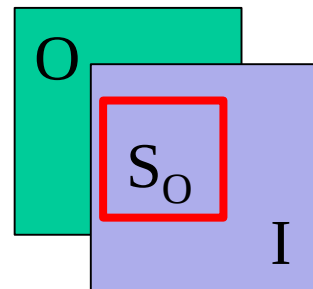
纹理合成

- **Graphcut Textures: Image and Video Synthesis Using Graph Cuts**

- 第一步：计算候选块的偏移量，有三种候选方法：

- 3. 子块差异度量：在输出图像中选取一部分像素计算距离平方和，较为通用，对水、烟纹理合成效果较好：

$$C(t) = \sum_{p \in S_O} |\mathbf{I}(p-t) - \mathbf{O}(p)|^2$$



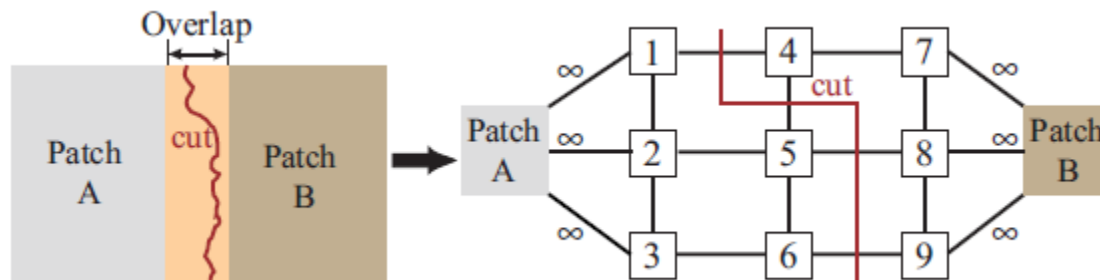


纹理合成

- **Graphcut Textures: Image and Video Synthesis Using Graph Cuts**

- 第二步：基于图割的候选块切割：

- 找出一条最好的割，使得该割线上的像素边的代价和最小，例如下图中的割的代价为：



$$M_{cut} = M(1,4, A, B) + M(5,4, A, B) + M(5,8, A, B) + M(6,9, A, B)$$

其中， $M(s, t, A, B) = \|A(s) - B(s)\| + \|A(t) - B(t)\|$



纹理合成

- 一些Graph-Cut算法的结果



Input



Graph cut



纹理合成

- 一些Graph-Cut算法的结果



Sample Texture



Synthesized Texture



本节课程内容

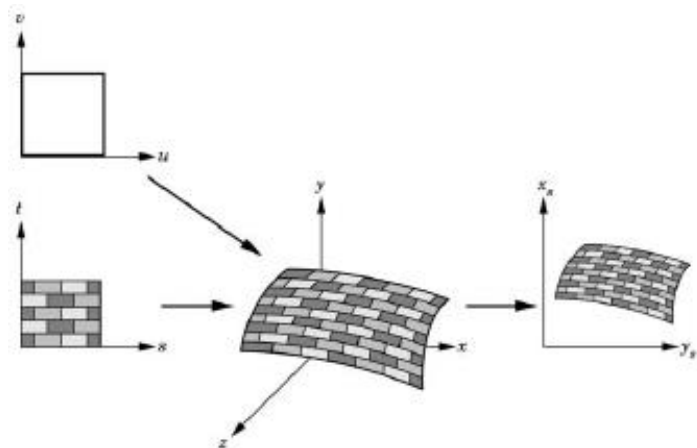
- 纹理简述
- 纹理采集、纹理合成
- 纹理映射
- 纹理滤波
- 纹理相关的高级主题



纹理映射

- 什么是纹理映射?

- 给定一个模型和2维纹理图像。
- 将**图像映射到模型**上:
 - 通过一个将模型点映射到 (u,v) 图像坐标的方程来完成。这个方程被称为 **表面映射函数(surface mapping function)**。
- 给模型上的点着色时, 我们从2维纹理中寻找适当的像素, 并用该像素的值去影响最终的颜色。





纹理映射

- 如何制定表面映射函数？
 - 通过自然参数化(**Natural parameterization**)
 - 对于规则物体，如球体、立方体、圆柱体...
 - 通过手工制定纹理坐标
 - 对于复杂物体，为每一个顶点指定一个纹理坐标。



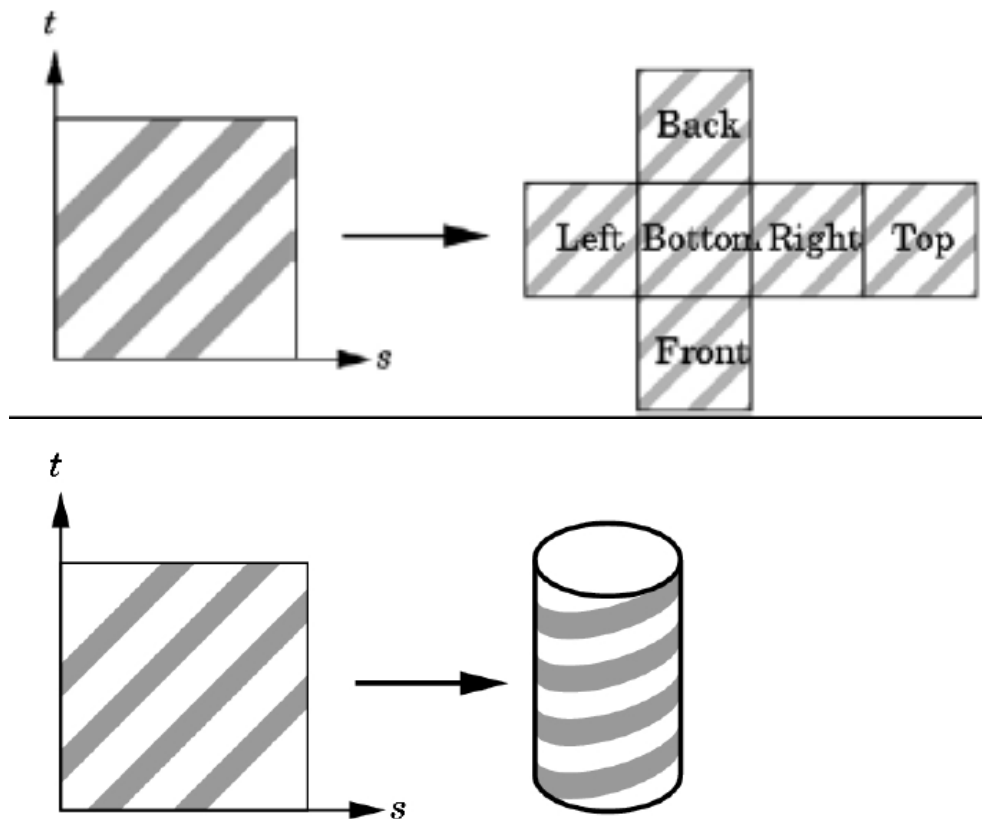
纹理映射

- 自然参数化
 - 球体:
 - 可以采用球面坐标 $(\theta, \phi) = (\pi u, 2\pi v)$
 - 圆柱:
 - 可以利用圆柱坐标 $(u, \theta) = (u, 2\pi v)$
 - 立方体
 - 每个面可以直接使用其面坐标。



纹理映射

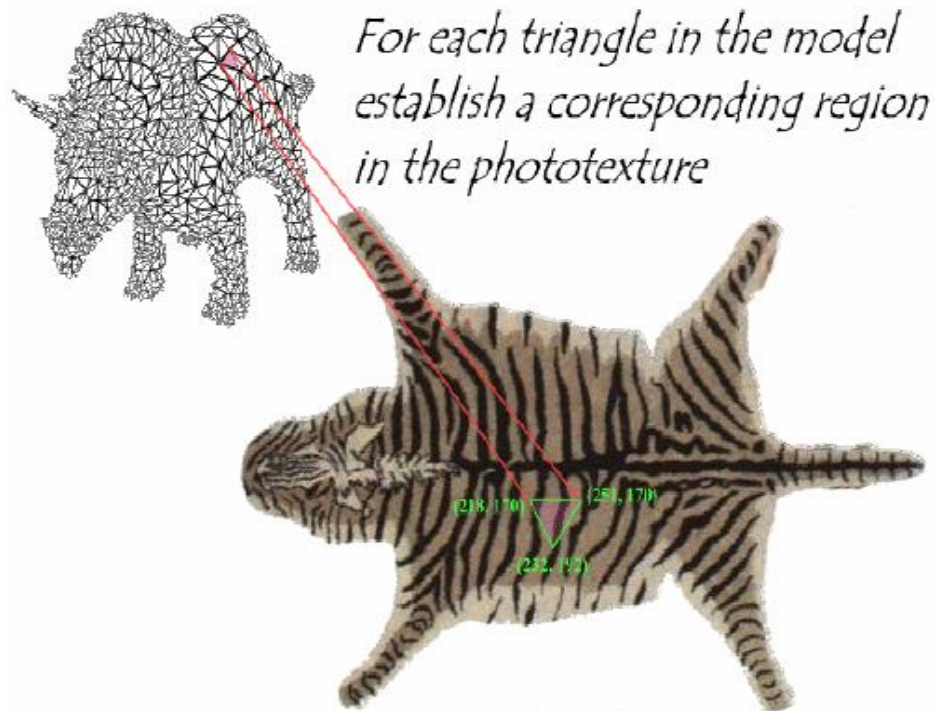
- 自然参数化





纹理映射

- 手工指定纹理坐标
 - 给每个顶点指定一个纹理坐标
 - 将图像空间中的一个三角形映射到物体空间。



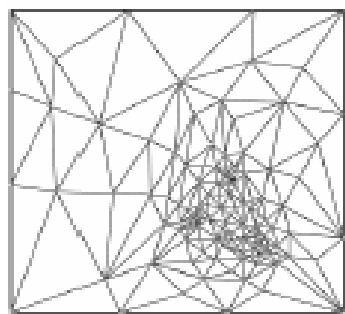
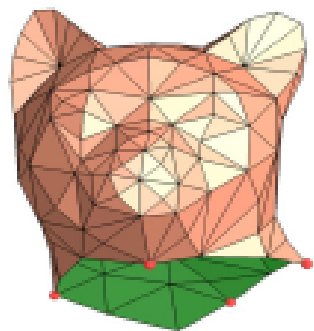
*For each triangle in the model
establish a corresponding region
in the phototexture*

*During rasterization interpolate the
coordinate indices into the texture map*

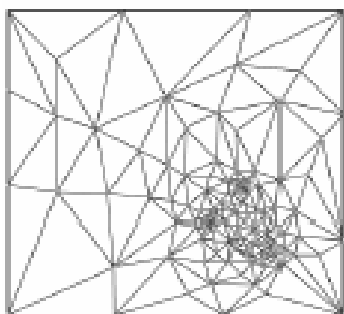


• 网格参数化

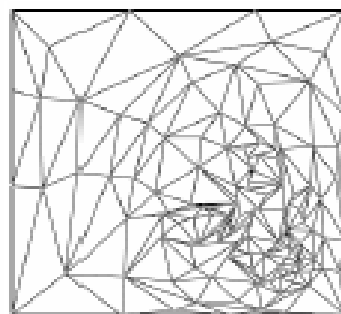
- 构造一个从三维模型到平面域的映射



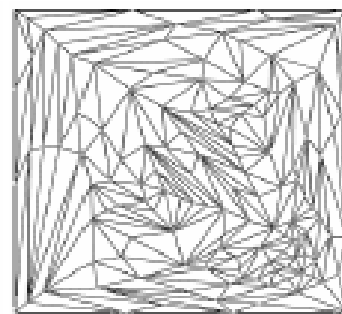
Floater



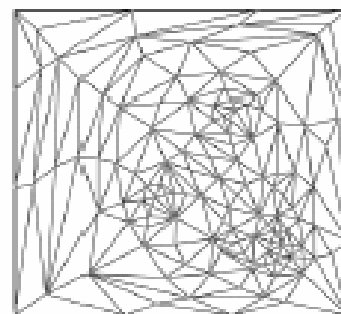
MIPS



Maillot



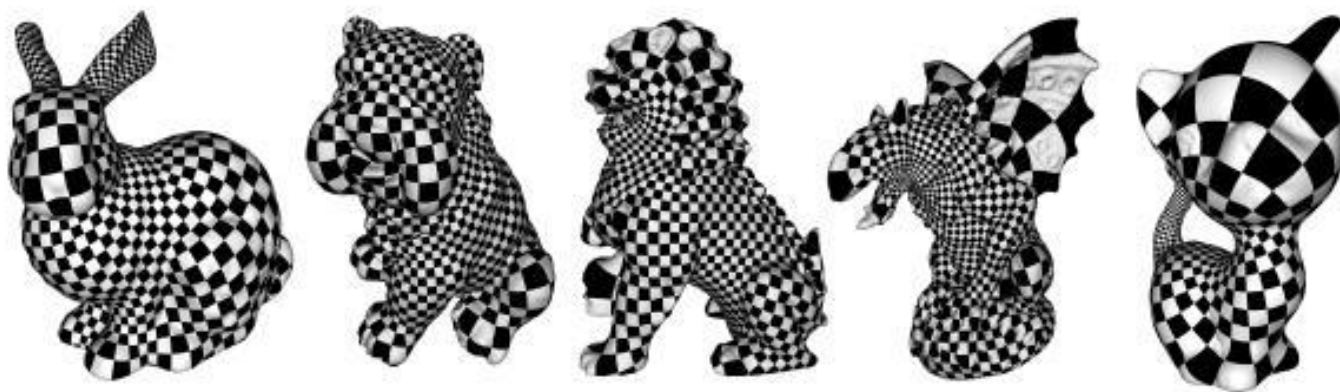
area-preserving



Sander



- 共形参数化与面积保持
 - 共形参数化要求保持直角

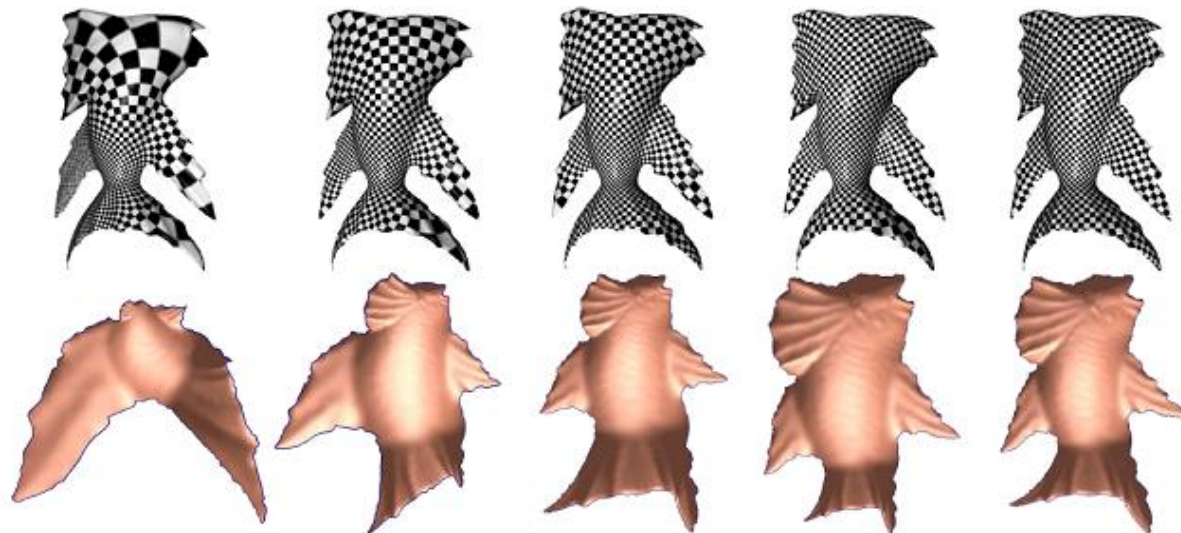


- Xianfeng Gu, Shing-Tung Yau, Global conformal surface parameterization, SGP 2003 (593 cites)



- 共形参数化与面积保持

- 保面积参数化：逆曲率映射



- Yong-Liang Yang, Junho Kim, Feng Luo, Shi-Min Hu, and Xianfeng Gu, Optimal Surface Parameterization Using Inverse Curvature Map , IEEE Transactions on VCG, 2008 (63 citation)



本节课程内容

- 纹理简述
- 纹理采集、纹理合成
- 纹理映射
- 纹理滤波
- 纹理相关的高级主题
- 卷积神经网络基础



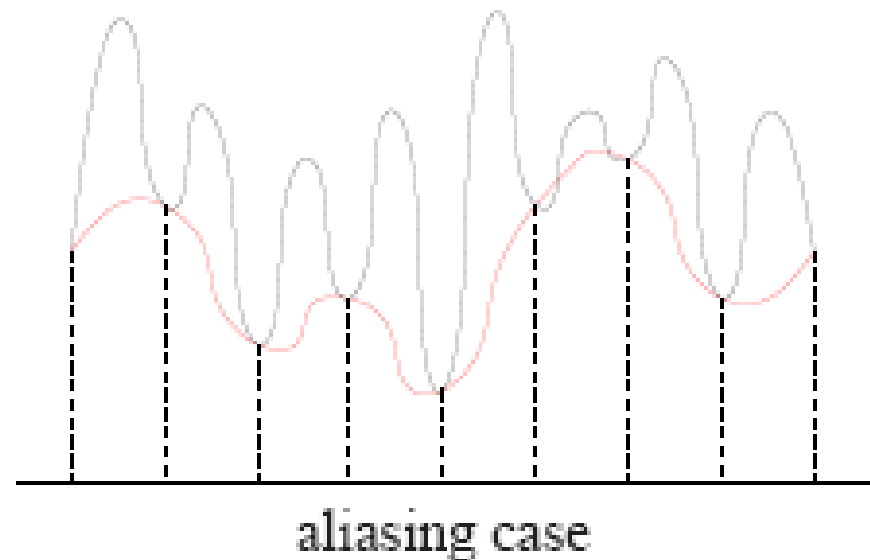
纹理滤波(Texture Filtering)

- 现在我们已经知道如何产生纹理以及如何将其映射到一个物体上。下一步就是渲染它。
- 然而，如果仅是直接渲染，就有可能产生人为痕迹---伪影(artifacts)。这些伪影由信号走样(signal aliasing)引起。



纹理滤波

- 走样(Aliasing)和反走样(Anti-aliasing)
 - 通常情况下，走样是由于信号采样频率过低产生的。因而导致信号的很多高频特征丢失。
 - 例如：右图中黑线表示原始信号，红线表示走样的信号。





纹理滤波

- 回忆第一节习题课。要解决走样问题，通常的方法包括：
 - 提高采样率。然而，该方案并不总是可行。例如，在图形学应用中，屏幕分辨率限制了采样频率。
 - 预滤波。即预先对信号进行滤波，得到一个更低频率的信号。



纹理滤波

- 各向同性的 (Isotropic) 滤波
 - 多级渐远纹理(Mipmapping):
 - 构造由一组经过预滤波和重采样图像构成的图像金字塔。这些图像是原图在 $1/2$, $1/4$, $1/8$ 等尺度对应的采样。
 - 在光栅化(rasterization)过程中, 我们计算与期望采样率最相近的图像的序号。



纹理滤波

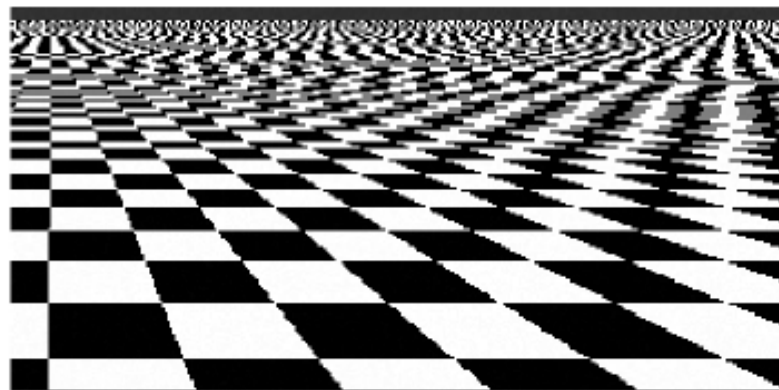
- 多级渐远纹理(Mipmapping)示例



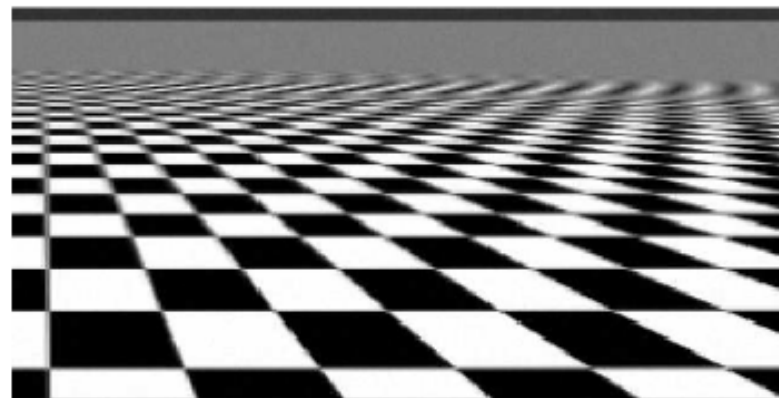


纹理滤波

- 利用多级渐远纹理进行反走样



aliasing



isotropic filtering



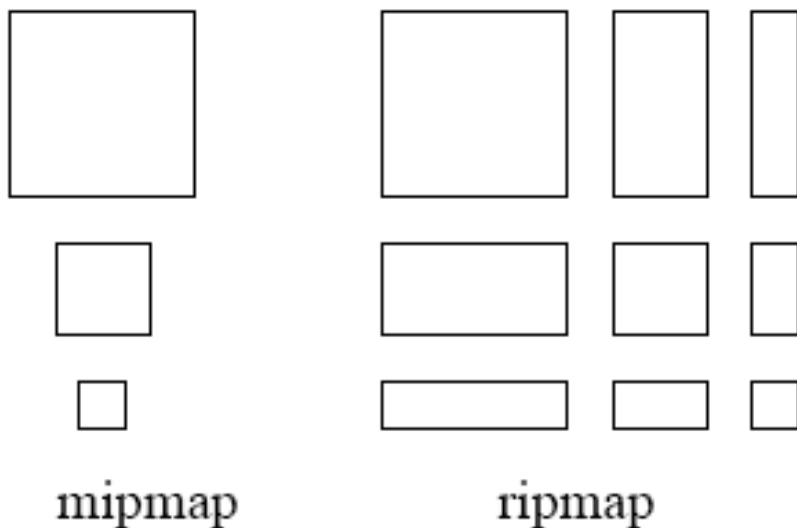
纹理滤波

- 各向异性 (Anisotropic) 滤波
 - 然而，多级渐远纹理的问题是它有时候会带来过模糊(over-blur)。正如上一页示例中所显示的，在我们建立多级纹理时，我们通常将各向同性的对高分辨率纹理进行滤波。这种方法在期望的滤波器是各向异性(透视情况)的时候会引起过模糊。



纹理滤波

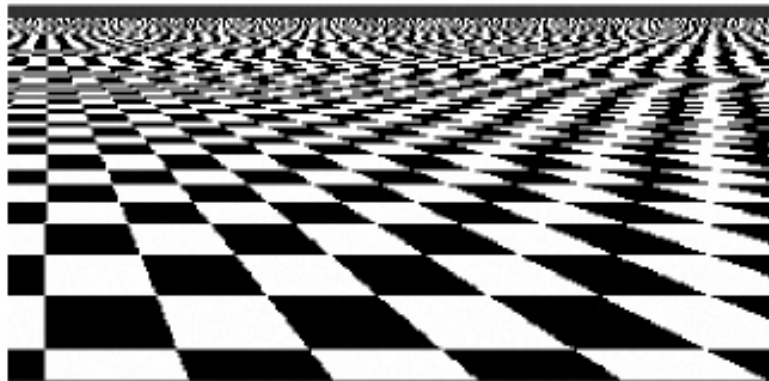
- 各向异性 (Anisotropic) 滤波
 - 通过预计算各项异性的滤波构造一个 **ripmap** 来代替 **mipmap** (多级纹理),



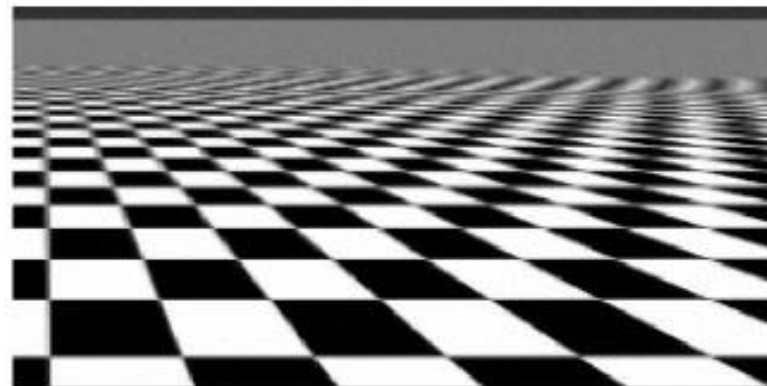


纹理滤波

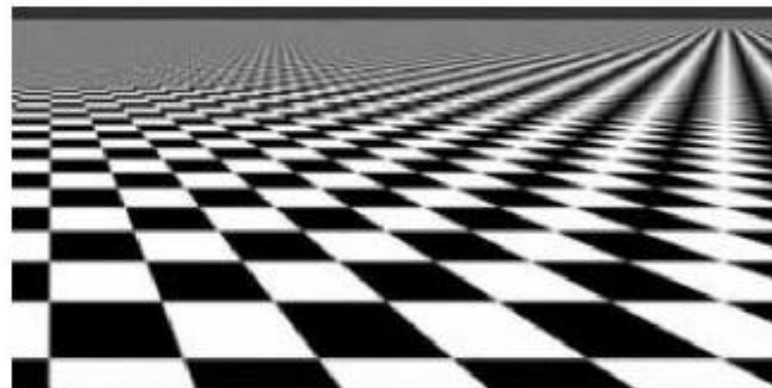
- 各向异性 (Anisotropic) 滤波



aliasing



isotropic filtering



anisotropic filtering



共形参数化在纹理的那个步骤中被经常使用?

- ☐ A 纹理采集
- ☒ B 纹理贴图/映射
- ☐ C 纹理滤波

提交



本节课程内容

- 纹理简述
- 纹理采集、纹理合成
- 纹理映射
- 纹理滤波
- 纹理相关的高级主题
- 卷积神经网络基础



纹理相关的高级主题

- 体纹理(Solid Textures)
- 凹凸贴图(Bump mapping)
- 移位贴图(Displacement mapping)
- 基于深度神经网络的纹理方法



纹理相关的高级主题

- 体纹理
 - 与二维图像纹理相对，通过三维体素定义纹理。
 - 为纹理到物体的映射创建一个三维参数化。





纹理相关的高级主题

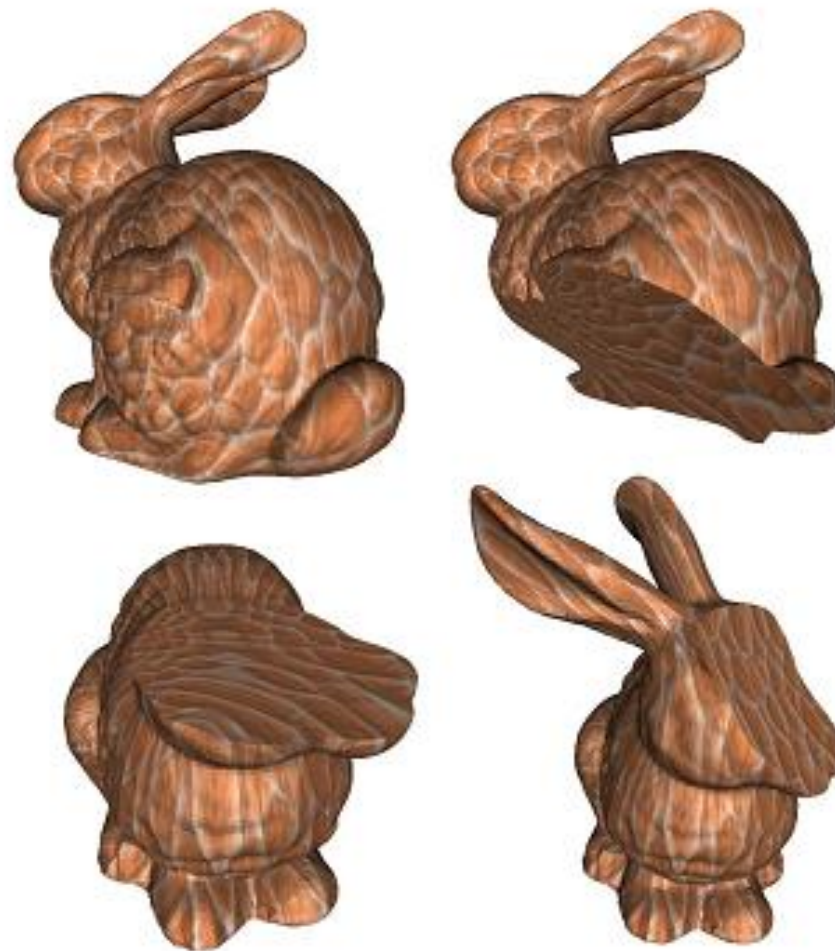
- 体纹理

- 纹理值构成的三维数组(例如：一个大理石块).
 - 利用一个 $[xyz] \rightarrow [RGB]$ 的函数把颜色映射到点上。
- 这种三维纹理被称为体纹理。
- 实际使用时，这种映射通常用过程定义，不需要存储整个三维颜色数组。
 - 仅需定义一个为每个三维点产生颜色的函数。



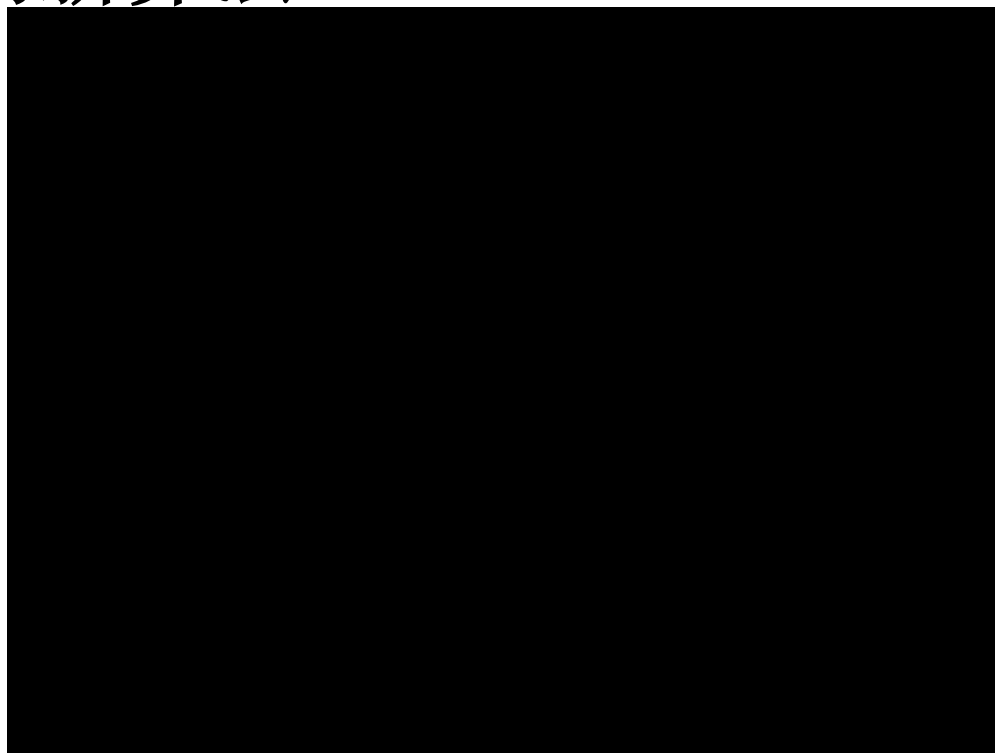
纹理相关的高级主题

- 体纹理示例





- 体纹理生成的新算法



- Song-pei Du, Shi-Min Hu, Ralph Martin, Semi-Regular Solid Texturing from 2D Exemplars, IEEE TVCG 2013.



纹理相关的高级主题

- 除颜色以外更多的映射类型
 - 基本的纹理映射给曲面赋予颜色
 - 但是方法本身更具一般性
 - 可以将更多种类的数据映射到曲面
 - 我们今天将讨论一组例子
 - 凹凸贴图(Bump mapping)
 - 移位贴图(Displacement mapping)



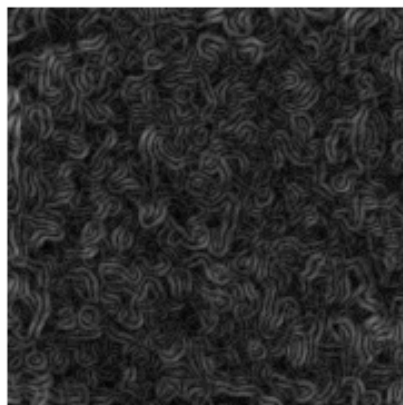
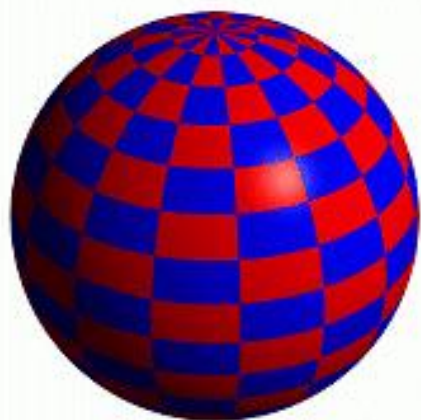
纹理相关的高级主题

- 除颜色以外更多的映射类型
 - 利用纹理来影响以下参数
 - 表面颜色 - 普通纹理
 - 表面法向 - 凹凸贴图 (**Blinn 1978**)
 - 几何 - 移位贴图
 - 光源辉度 – 环境贴图 (**Blinn 1978**)



纹理相关的高级主题

- 凹凸贴图
 - 用纹理改变表面法向
 - 并不改变表面的实际形状
 - 只是在渲染的时候好像形状改变了



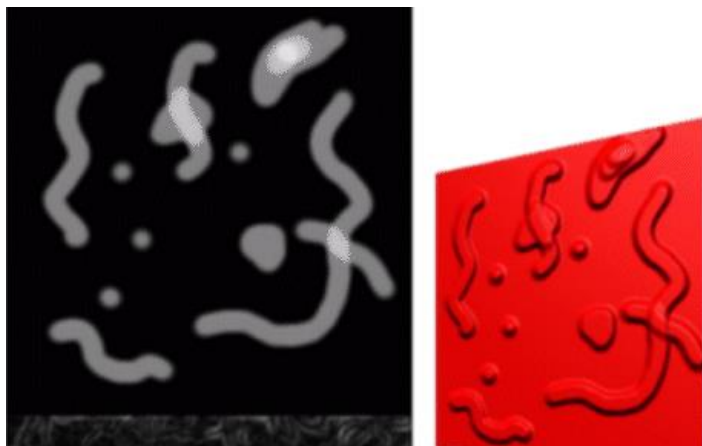


纹理相关的高级主题

- 凹凸贴图

- 把纹理图像当成法向方向上位置的扰动的高度方程 $F(u,v)$

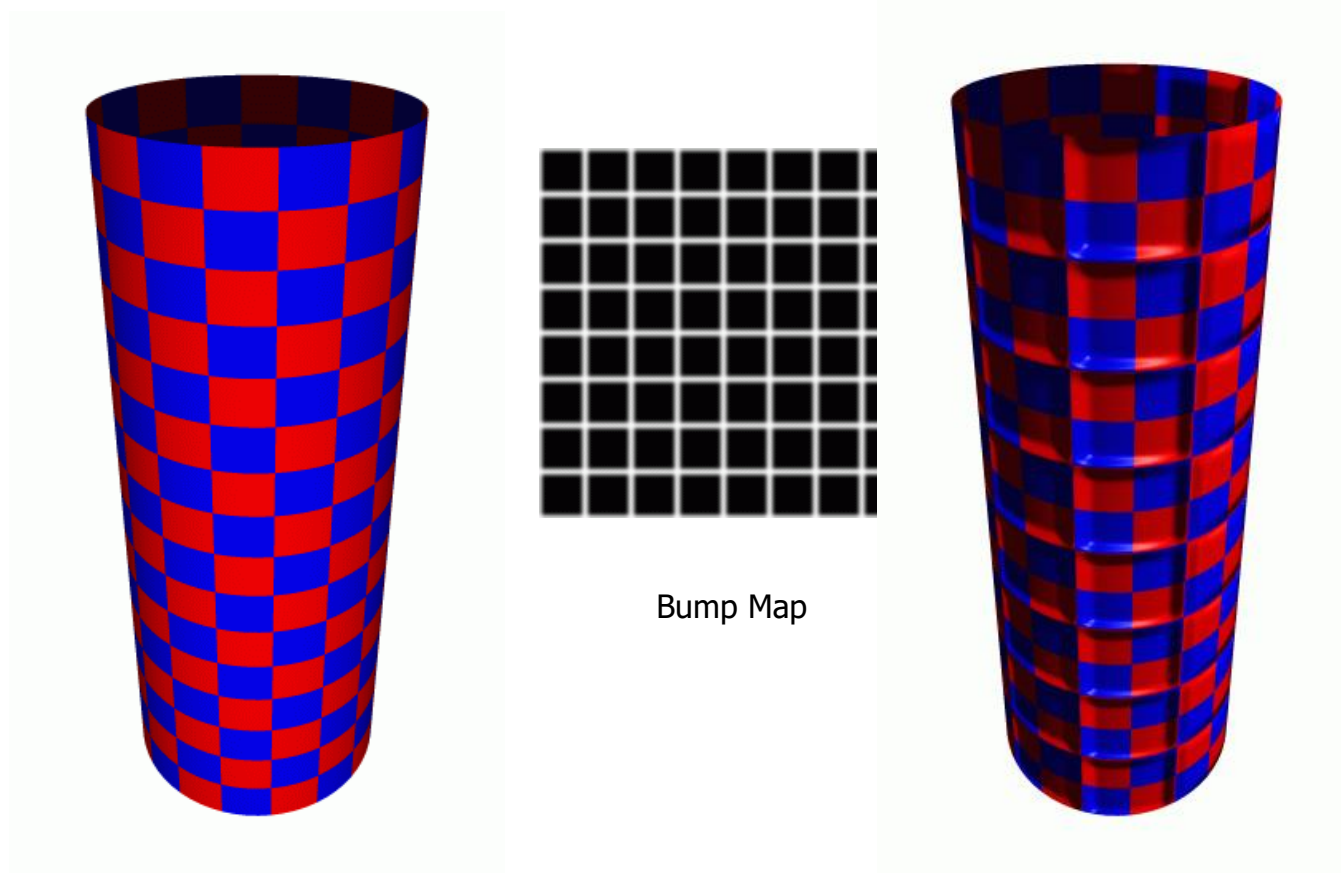
$$\tilde{P}(u,v) = P(u,v) + F(u,v) * N(u,v)$$





纹理相关的高级主题

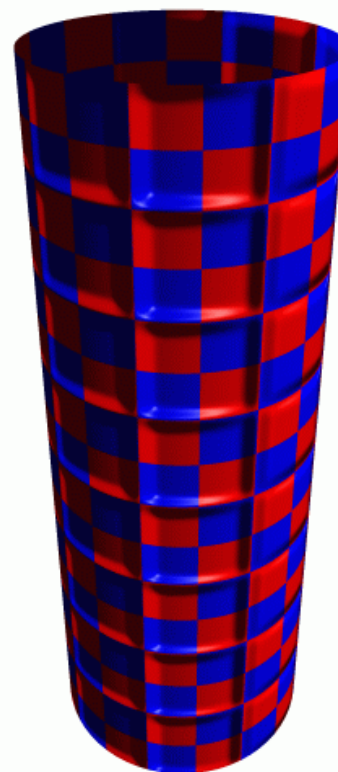
- 另一个凹凸贴图(Bump Mapping)示例





纹理相关的高级主题

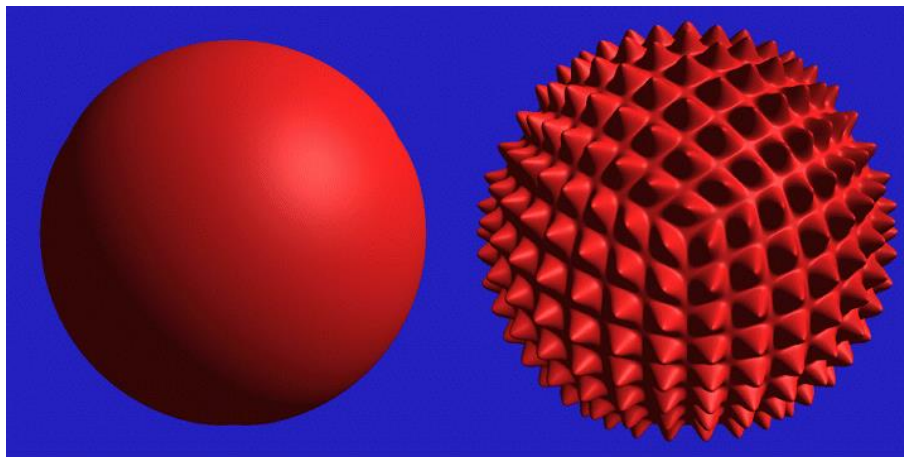
- 凹凸贴图
 - 但是，凹凸贴图不能产生侧面轮廓 (silhouette) 效果
 - 它不支持自遮挡或者自阴影





纹理相关的高级主题

- 移位贴图
 - 利用纹理图真实地移动曲面点
 - 在确定可见性(visibility)之前，必须先对几何进行移位
 - 必须在预处理时进行，而不是硬件渲染时





基于深度学习的纹理合成

- Non-stationary Texture Synthesis by Adversarial Expansion (ACM SIGGRAPH'18)
- 给定一个 $h \times w$ 的纹理图片，通过深度神经网络模型映射到一个 $2h \times 2w$ 的图片





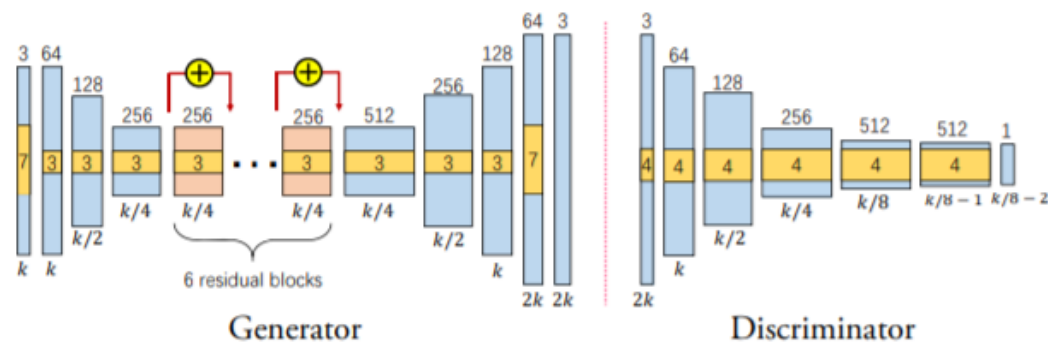
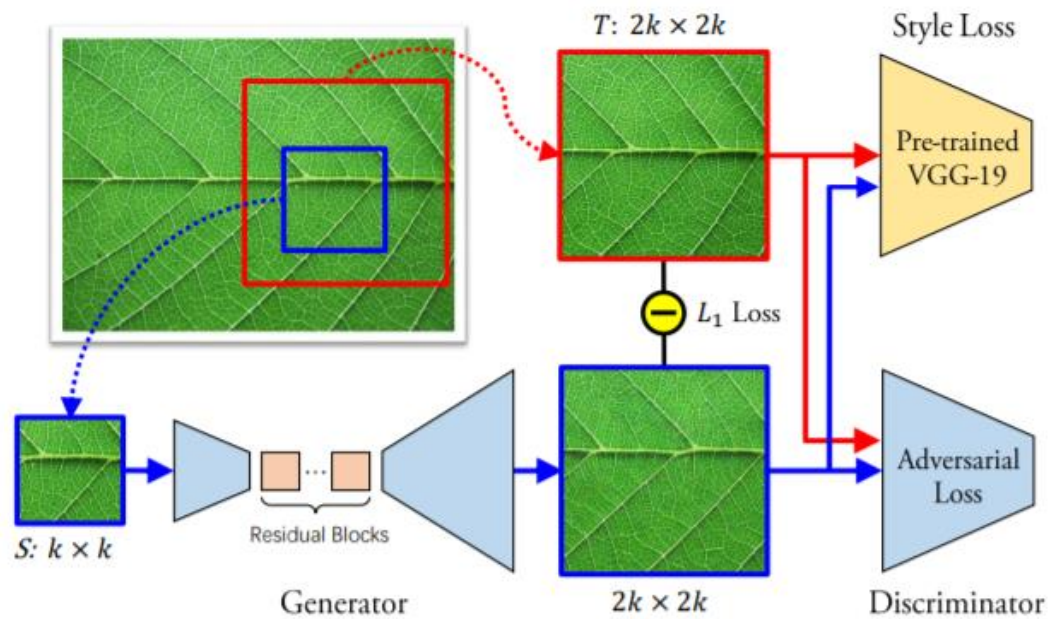
基于深度学习的纹理合成

- 该方法可以保持大尺度上的纹理结构





基于深度学习的纹理合成





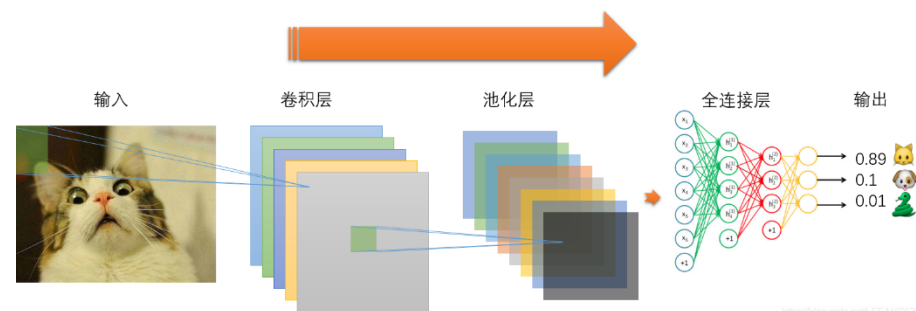
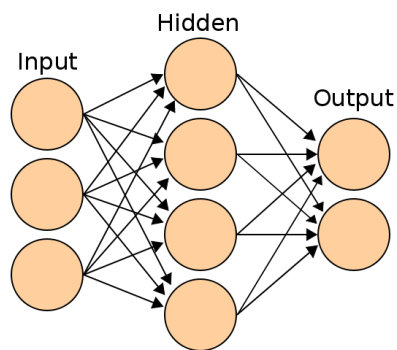
本节课程内容

- 纹理简述
- 纹理采集、纹理合成
- 纹理映射
- 纹理滤波
- 纹理相关的高级主题
- 卷积神经网络基础



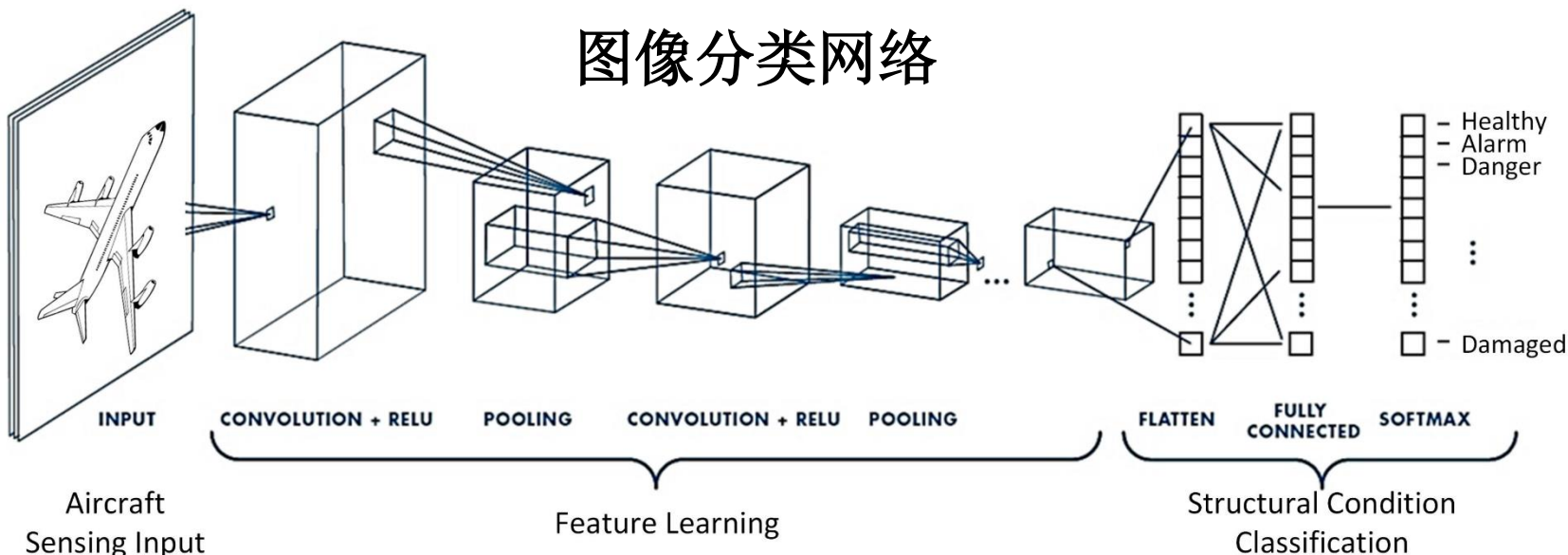
深度神经网络

- 人工神经网络是由线性和非线性变换组合而成一个映射，从输入数据得到输出结果（如分类概率）
- 早期神经网络只有一个隐含层
- 深度神经网络增加了隐含层数
- 针对图像的深度神经网络引入了一类特殊的线性变换——卷积





卷积神经网络总览



网络模块:

1. 卷积运算
2. 非线性变换
3. 池化
4. 全连接层
5. 损失函数

训练过程: 输入图像, 输出类别概率

- 提取图像特征, 特征分类
- 根据预测概率和真实标签计算损失
- 根据损失求解梯度, 更新网络参数



卷积运算

- 数学上的卷积定义
- 设 f, g 是两个函数，用 $*$ 表示卷积运算，则 f 与 g 的卷积 $h = f * g$ 仍为一个函数，函数值按如下方式定义
- 一维情形：

$$h(x) = \int_{-\infty}^{+\infty} f(u)g(x-u)du$$

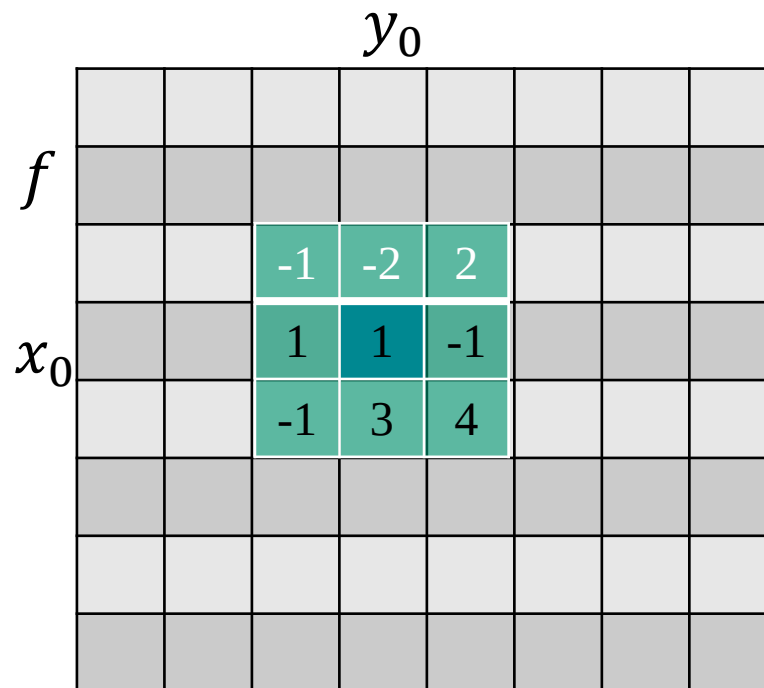
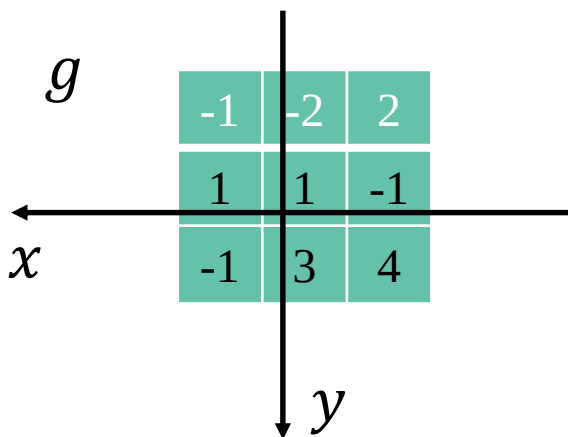
- 二维情形

$$h(x, y) = \int_{-\infty}^{+\infty} \int_{-\infty}^{+\infty} f(u, v)g(x-u, y-v)dudv$$



卷积运算

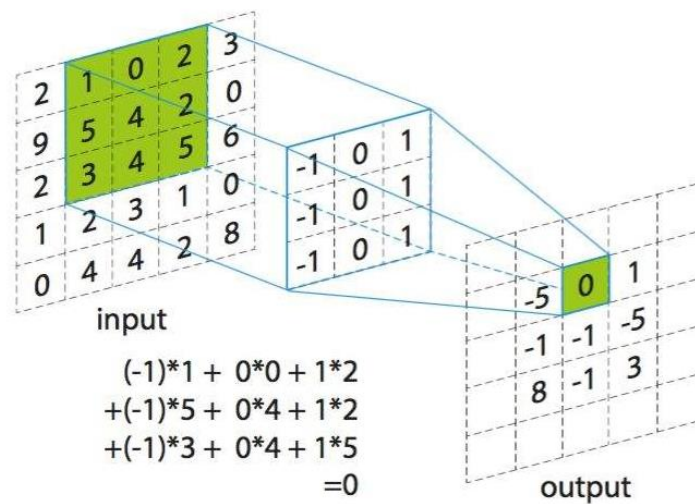
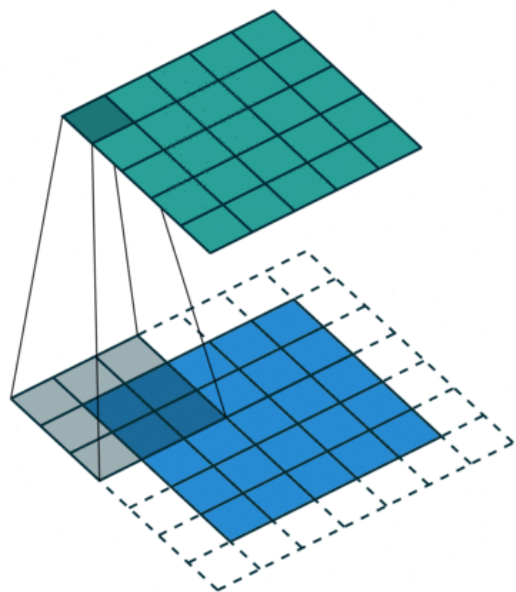
- 将卷积核按 x 轴和 y 轴翻转
- 对 f 上的位置 (x_0, y_0) ，将卷积核中心覆盖上去，对每个位置相乘后求和，得到该点的卷积结果
- 遍历所有的位置完成卷积





卷积运算

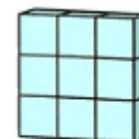
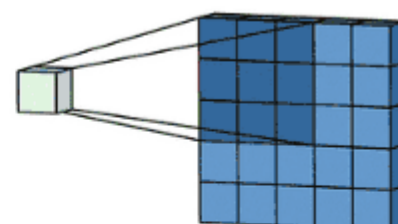
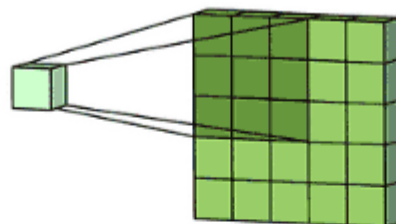
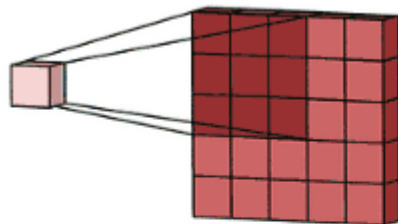
- 在实际的计算过程中，我们不对卷积核 g 进行翻转，直接覆盖到 f 进行扫描和相乘求和，但我们通常仍将这种操作称为“卷积”





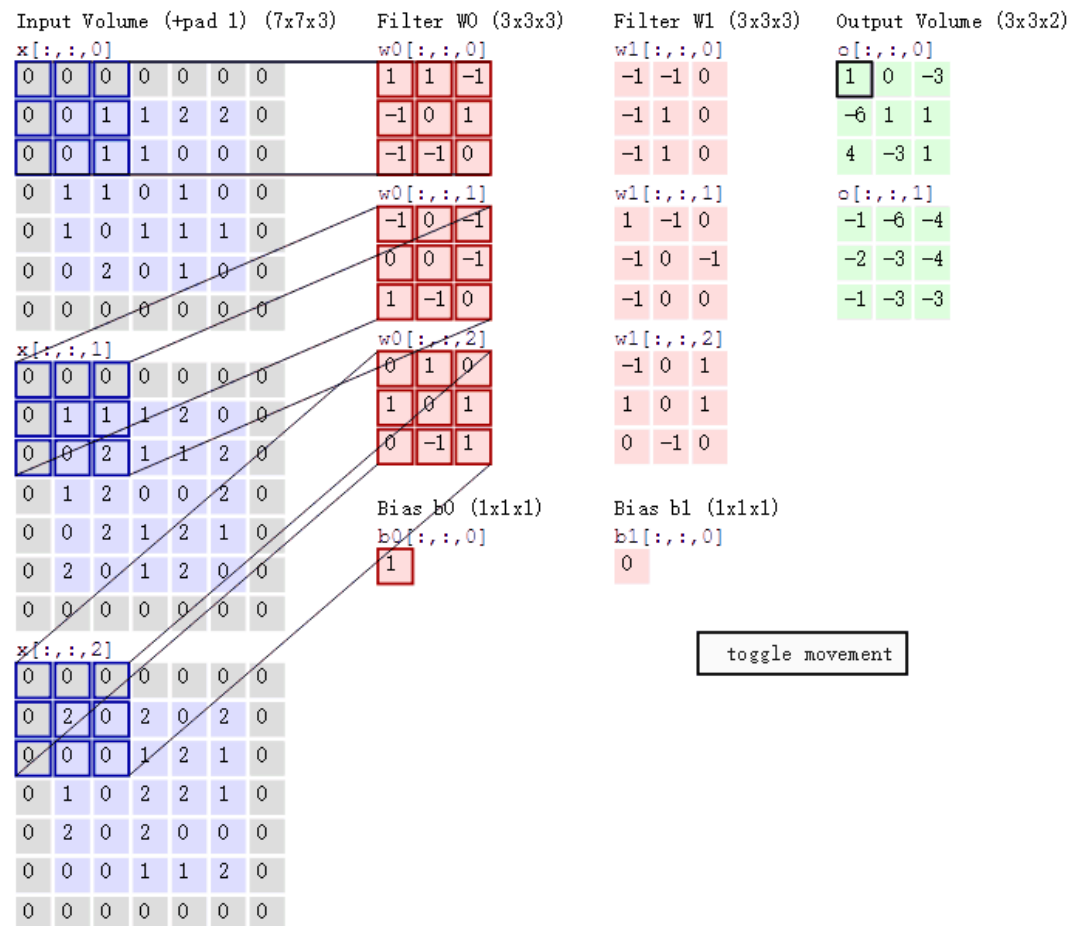
卷积运算

- 多通道卷积示意图





卷积运算





图像处理中常见的卷积核

- Sobel梯度算子



$$g = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$



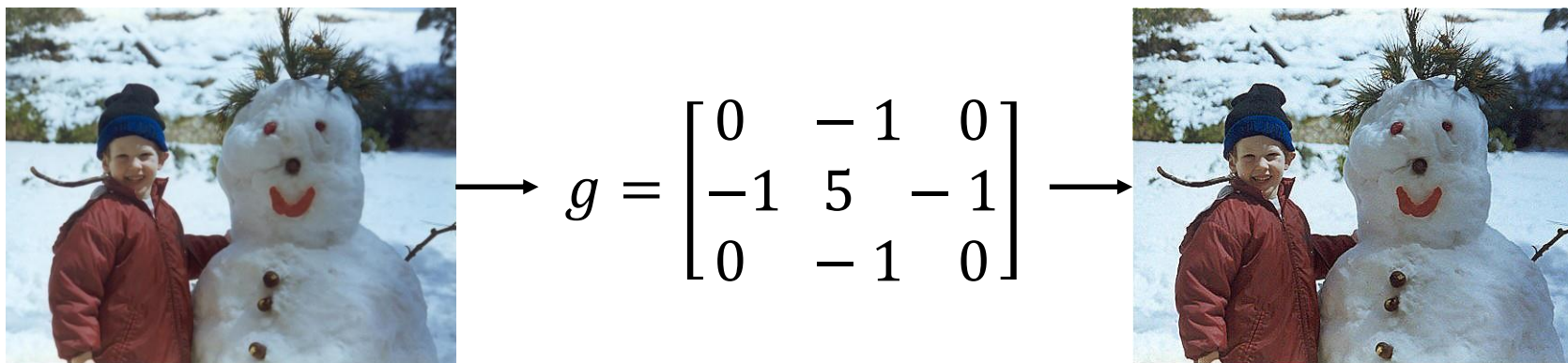
$$g = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$





图像处理中常见的卷积核

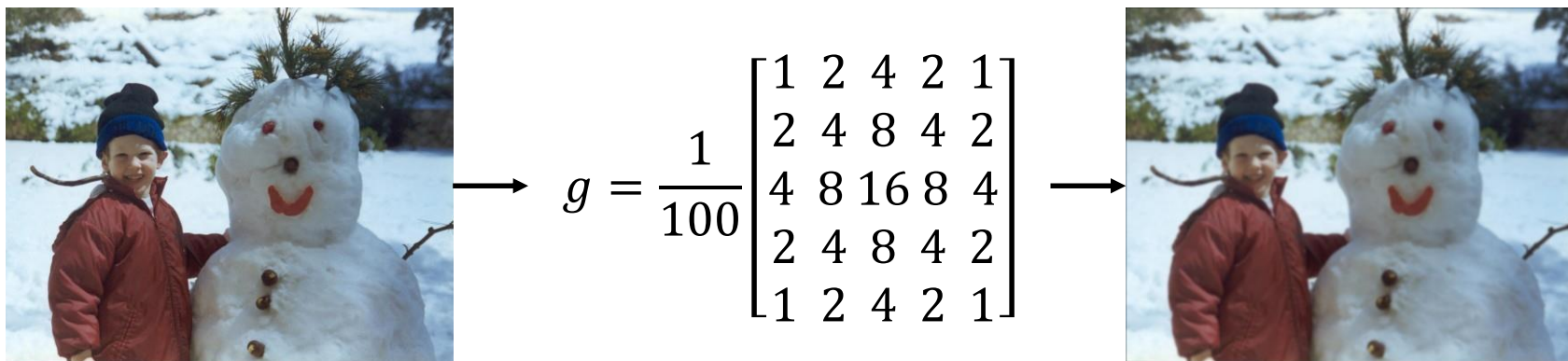
- Laplacian边缘锐化





图像处理中常见的卷积核

- Gaussian模糊





卷积运算

- 对图像施加不同的卷积核，就可以得到不同的图像特征（feature map），这些特征可以用于识别、分类等任务
- 在神经网络中，我们把卷积核的权重视为待学习的模型参数
- 对于 $H \times W$ 的输入图片或中间层特征，假设其通道数为 C ，那么一个卷积核包含 $C \times k \times k$ 个参数，分别对每个通道用 $k \times k$ 的核做卷积再求和



非线性变换

- 如果把图像上的所有像素值线性排列为一个向量 x ，卷积操作可以视为一个矩阵乘法 $x' \rightarrow Wx$
- 简单将两个卷积操作叠加，得到的仍然只是一个线性变换： $x \rightarrow W_1x \rightarrow W_2(W_1x) = (W_2W_1)x$
- 因此我们要在卷积运算之间增加某种非线性变换 f :

$$x \rightarrow W_1x \rightarrow f(W_1x) \rightarrow W_2f(W_1x) \rightarrow \dots$$

- 交替使用卷积和非线性变换，可以构造出表达能力十分强大的模型

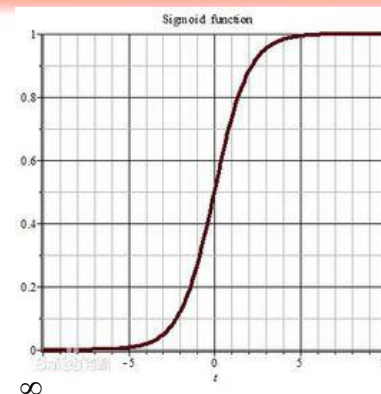


非线性变换

- 通常采用的非线性函数 f 有

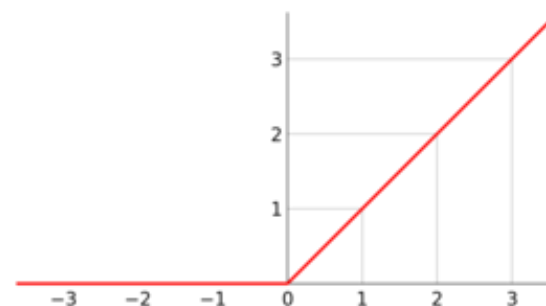
- Sigmoid函数: $f(x) = \frac{1}{1+e^{-x}}$

Sigmoid 能够把任意输入压缩到(0,1)之间



- ReLU函数: $f(x) = \max(0, x)$

ReLU则会把输入映射到 $[0, +\infty)$

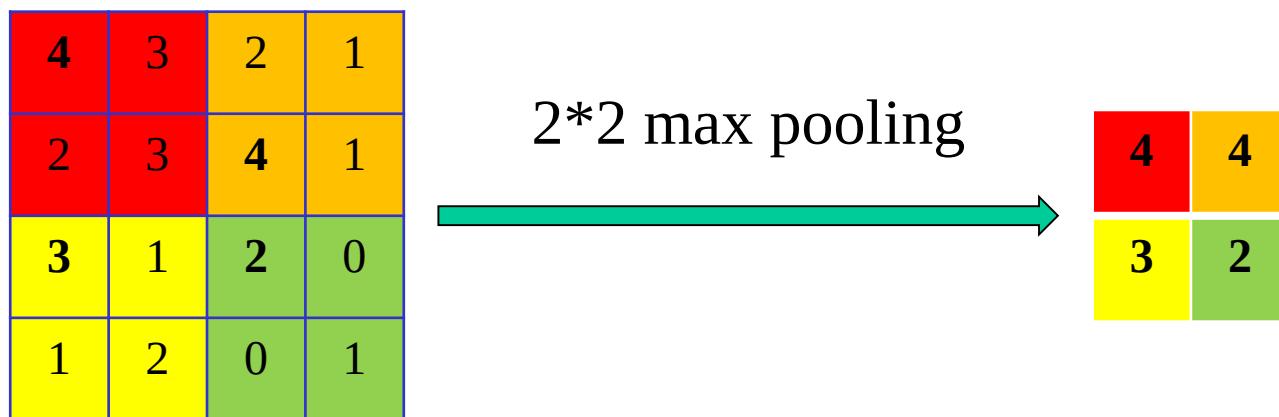


- 其他的非线性函数: Tanh, ELU, LeakyReLU,



池化操作

- 为了减小计算量及消除物体平移变换造成的影响，引入池化（pooling）操作，将输入的feature map上每一个 $k \times k$ 区域的值压缩成一个值
 - 区域内取最大值：max pooling
 - 区域内取平均值：average pooling





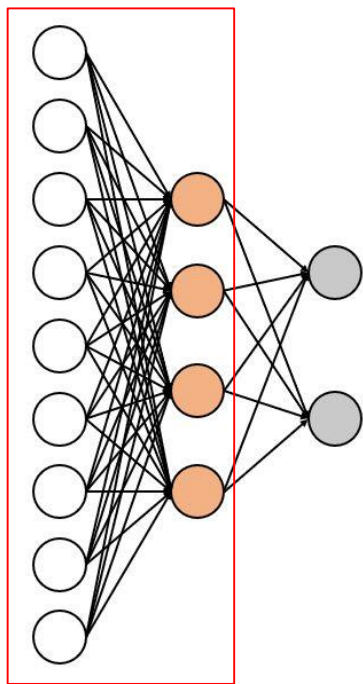
全连接层

- 卷积操作可以视为一个矩阵乘法 $x \rightarrow Wx$
- 如果不考虑像素的位置信息，将所有像素上的数值排列成向量 $x \in R^n$ ，将 $W \in R^{m \times n}$ 中所有元素视为相互独立的自由变量，我们将这样的操作称为全连接层



全连接层

- 全连接的含义：输出 $x' = Wx$ 中的每个分量都和 x 的所有分量相关



全连接层：每一个神经元都和前面一层所有的神经元连接



概率归一化

- 最终全连接层输出每一类的概率，但是无法保证概率之和为1。
- 使用Softmax来实现概率归一化

$$p_c = \frac{e^{x_c}}{\sum_{i=1}^C e^{x_i}}$$

C 表示类别的数量

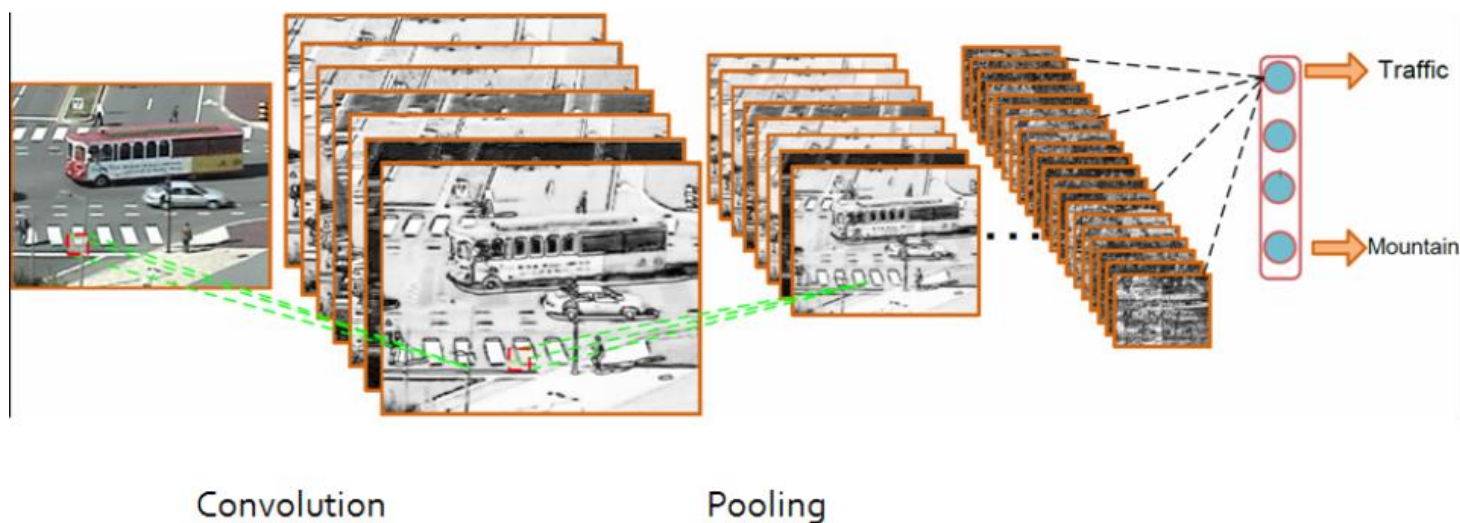
x_c 表示全连接层输出的属于 c 的概率

p_c 表示归一化后类别 c 的概率



卷积神经网络的基本结构

- 用于图像分类的卷积神经网络一般由两部分组成
- 特征抽取：使用若干轮卷积层——非线性层——池化层提取图片的语义特征
- 分类器：使用若干轮全连接层——非线性层将语义特征映射到图片所属类别的概率分布

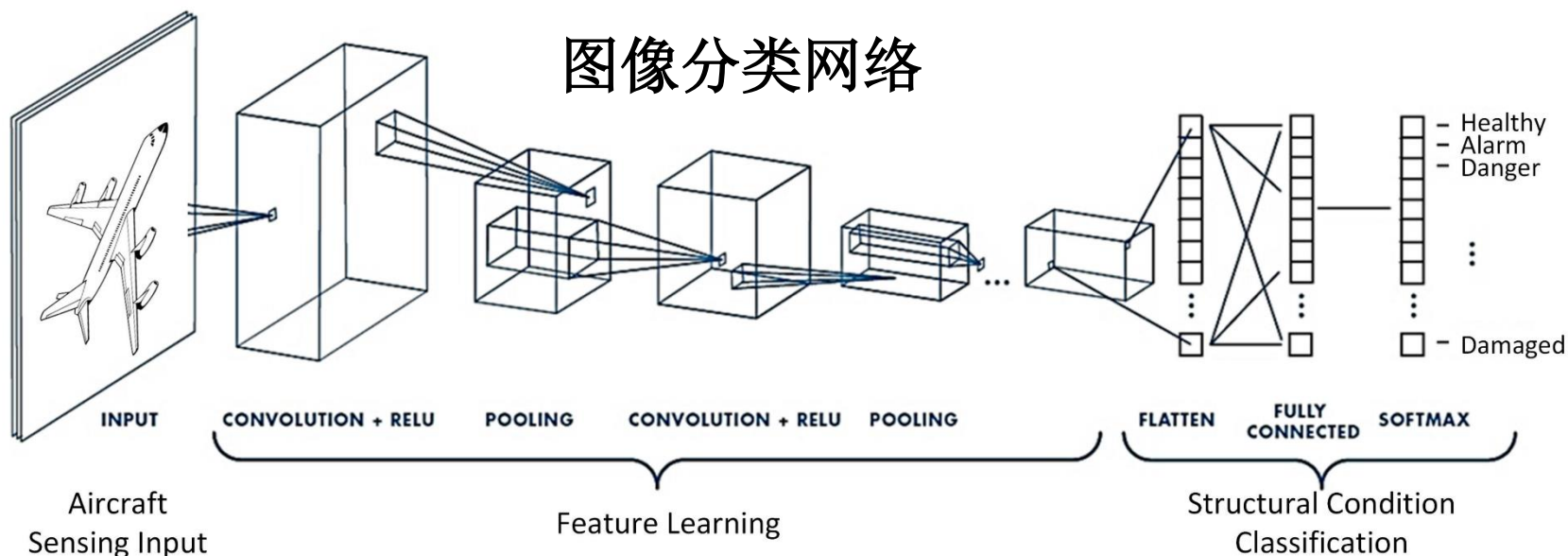




卷积神经网络的训练

- 由卷积层，非线性层，全连接层等组合成的网络整体上可视为一个函数 F ，其中待定的全体参数记为 θ
- 由输入图片 x 计算得到的图像类别概率为 $p = F(x; \theta)$ ，将其与目标分布 y 的交叉熵定义为损失函数： $L = cross_entropy(p, y)$
- 要使误差 L 达到最小，计算导数 $\partial L / \partial \theta$ ，并使用梯度下降等方法更新参数 θ

总结



网络模块:

1. 卷积运算
2. 非线性变换
3. 池化
4. 全连接层
5. 损失函数

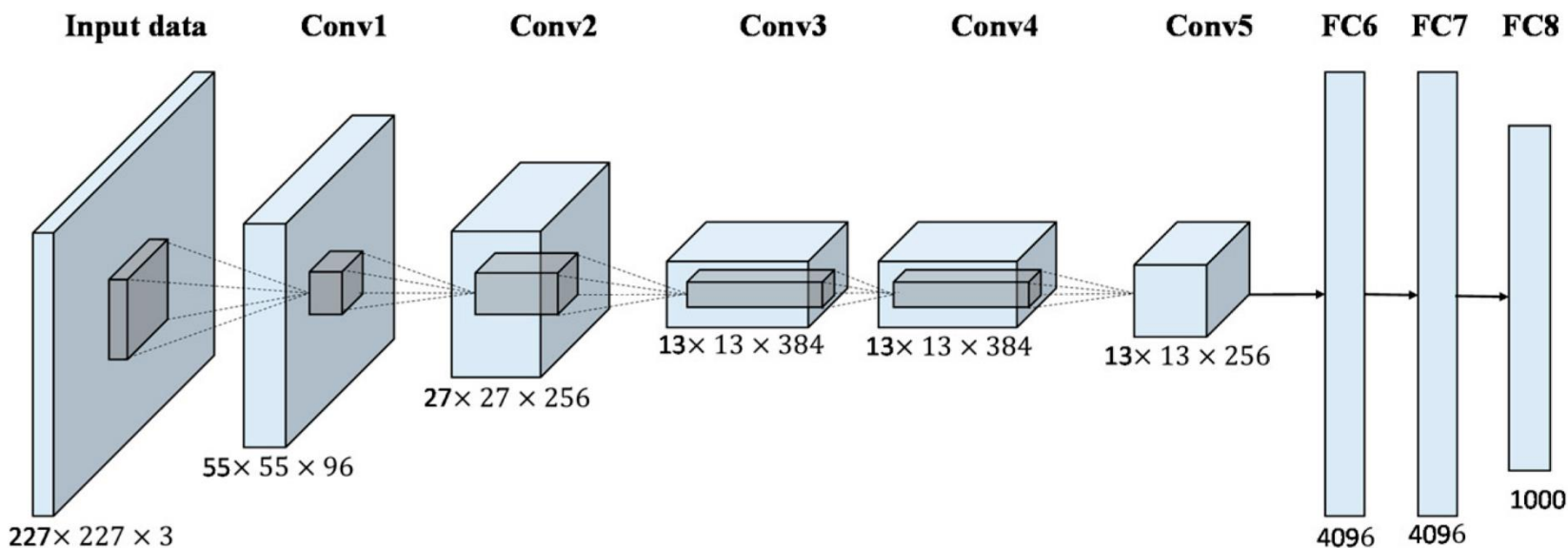
训练过程: 输入图像, 输出类别概率

- 提取图像特征, 特征分类
- 根据预测概率和真实标签计算损失
- 根据损失求解梯度, 更新网络参数



AlexNet

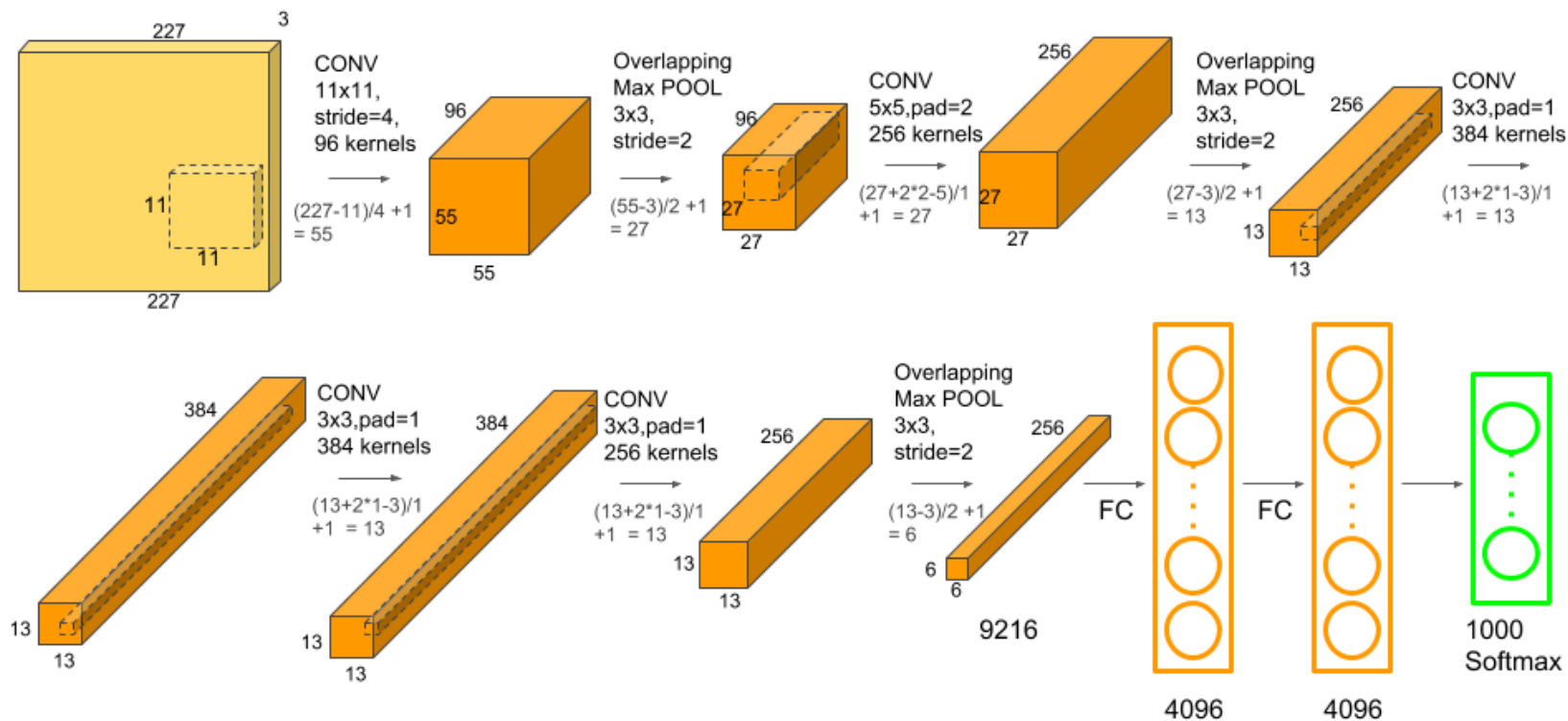
- 由Alex Krizhevsky在NIPS2012的论文ImageNet Classification with Deep Convolutional Neural Networks中提出
- 计算过程中，分辨率逐渐降低，特征维度逐渐增大
- AlexNet在ILSVRC-2012竞赛中获得了top-5测试的15.3%错误率，比第二名低10.9%，差距非常大，AlexNet在当时给学术界和工业界带来了巨大影响



AlexNet



AlexNet的特征变化过程





生成对抗网络

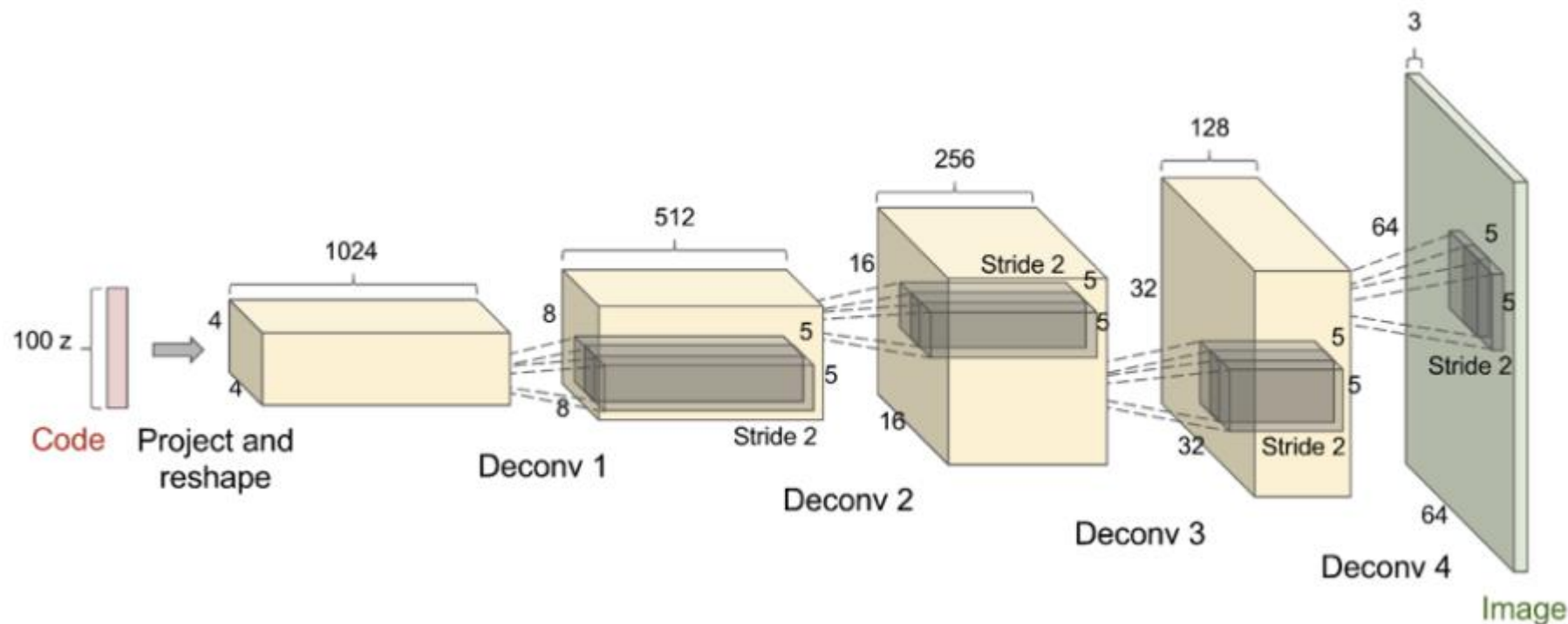
- Generative Adversarial Networks (GAN)
- 由Ian J. Goodfellow和Yoshua Bengio等在2014年提出
- 真实图片在整个图像空间中构成了一个子空间 $Y \subset R^{H \times W \times 3}$ ，我们假设这个子空间可以由 n 维向量 z 进行参数化表示
- 更进一步，我们假设 z 符合标准正态分布，并希望从参数 z 的空间还原出真实图片空间 Y





生成对抗网络

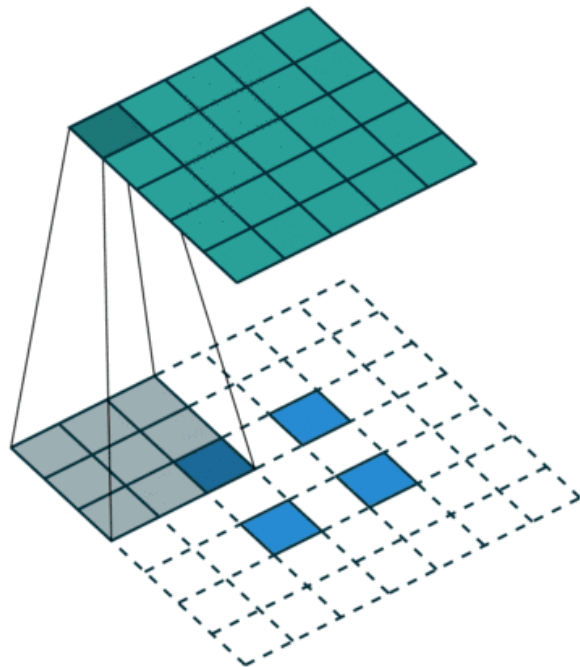
- 目标：从高斯分布中采样随机向量 z ，用神经网络 G 将向量 z 转换为图片
- 基本的生成器 G 架构
- 输入100维随机编码，输出 64×64 的RGB图像





生成对抗网络-反卷积

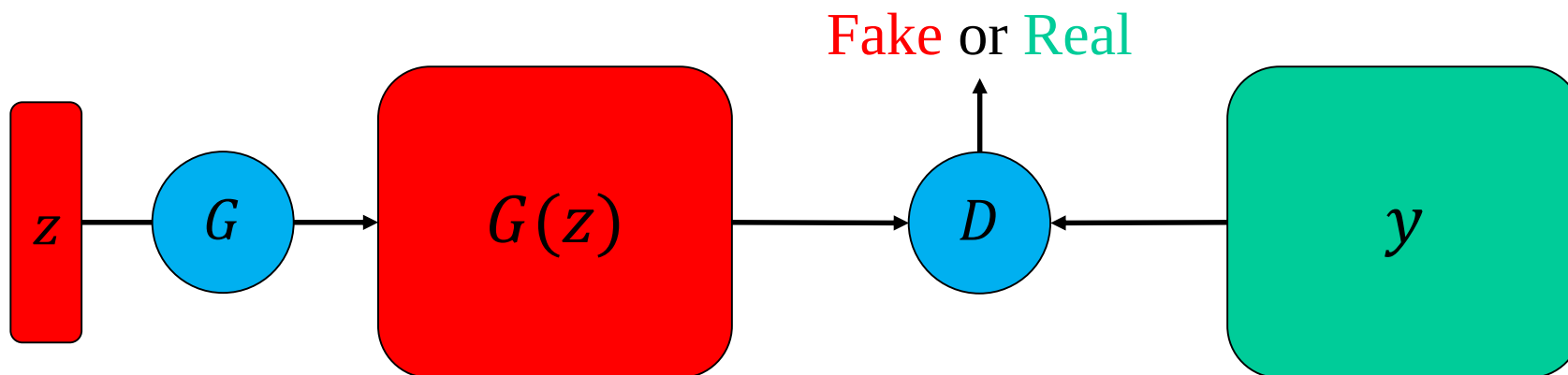
- 反卷积能够将小图转换为大图
- 具体过程如下图所示





生成对抗网络

- Generative Adversarial Networks (GAN)
- 为了保证图片的真实感，需要另外训练一个判别器 D ，来区分真实的图片 $y \in Y$ 和生成网络 G 创造的图片 $G(z)$





生成对抗网络

- 若设真实类别的标签为1，虚假类别的标签为0，对判别器 D 的输出结果计算交叉熵的相反数：

$$L(G, D) = \log D(y) + \log(1 - D(G(z)))$$

- 对判别器 D 而言， L 越大表示区分效果越好
- 对生成器 G 而言，要使生成的图片足够真实，就需要让判别器 D 无法正确区分，亦即 L 越小生成效果越好
- 对抗优化的目标

$$G^* = \arg \min_G \max_D L(G, D)$$



生成对抗网络

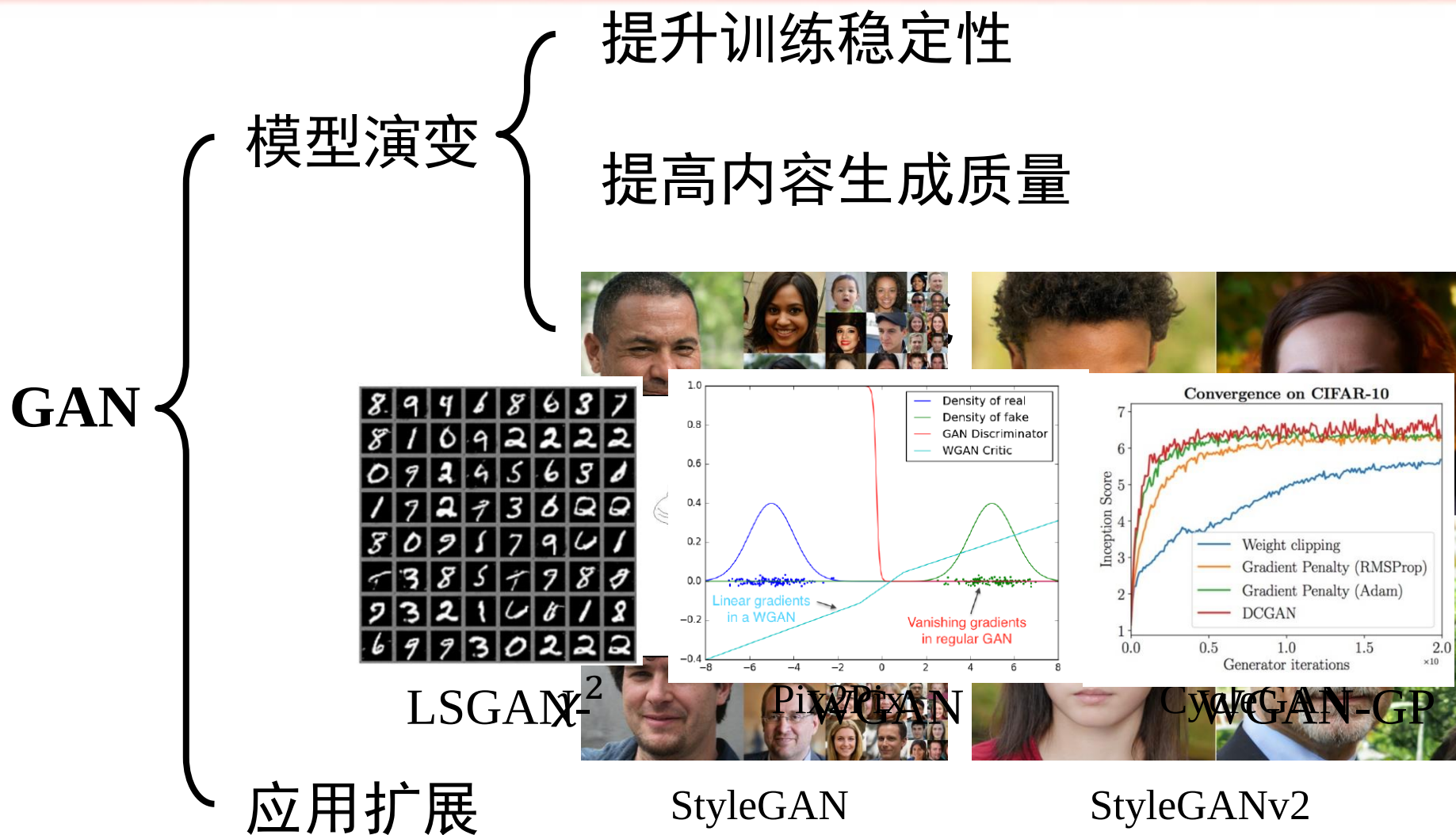
- 论文Generative Adversarial Nets中的结果
- 最右侧一列为训练数据，左侧为网络生成结果



Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David WardeFarley, Sherjil Ozair, Aaron Courville, Yoshua Bengio, Generative adversarial networks, NeurIPS 2014, (66221 cites)



生成对抗网络的演变与发展





今日人物: Alexei (Alyosha) Efros

- **Alexei Efros**, UC Berkeley教授
- 论文200+篇, 引用155737次(H111)
 - 2003年Berkeley博士
 - 导师: Jitendra Malik
 - 2003, 剑桥博士后
 - 2004年CMU助理教授
 - 成名作texture synthesis他引4869次
 - Pix2pix 26739, CycleGAN 27229
 - 2010获ACM SIGGRAPH杰出新人奖
 - **2017获ACM Computing Award (第一届)**





谢谢！